

Curso Profesional de Entity Framework Core 8

Módulo 5 - Persistencia empresarial

TEORÍA

AUTOR: JAIME GALLO

.NET 8 · Entity Framework Core 8 · Visual Studio Community · SQL Server LocalDB · Proyecto AceriaData

Índice

Punto 5.1 — Concurrencia optimista: concepto y necesidad	3
Punto 5.2 — Configuración de tokens de concurrencia	7
Punto 5.3 — Resolución de conflictos de concurrencia	11
Punto 5.4 — Transacciones: SaveChanges y transacciones explícitas	14
Punto 5.5 — Transacciones ambientales y buenas prácticas	18
Punto 5.6 — Migraciones en entornos de producción: estrategias y despliegue	22
Punto 5.7 — Migraciones idempotentes y scripts SQL	25
Punto 5.8 — Migraciones en equipos: conflictos y buenas prácticas	29
Punto 5.9 — Patrón Repositorio y Unidad de Trabajo en aplicaciones empresariales	32
Punto 5.10 — Logging y diagnóstico en Entity Framework Core	36
Punto 5.11 — Testing con EF Core	40
Punto 5.12 — Buenas prácticas y anti-patronos en persistencia empresarial	43

Módulo 5 — Persistencia empresarial: concurrencia, transacciones, despliegue, testing y buenas prácticas

12 puntos · 6 horas · AceriaData · .NET 8 · Entity Framework Core 8 · SQL Server LocalDB

Este módulo continúa AceriaData desde el estado final del Módulo 4 y se centra en los problemas de persistencia que aparecen cuando una aplicación pasa de consultar datos a operar en entornos concurrentes, desplegables y verificables.

Mapa del módulo

Punto	Tema	Duración de referencia
5.1	Concurrencia optimista: concepto y necesidad	30 min
5.2	Configuración de tokens de concurrencia	30 min
5.3	Resolución de conflictos de concurrencia	30 min
5.4	Transacciones: SaveChanges y transacciones explícitas	30 min
5.5	Transacciones ambientales y buenas prácticas	30 min
5.6	Migraciones en entornos de producción: estrategias y despliegue	30 min
5.7	Migraciones idempotentes y scripts SQL	30 min
5.8	Migraciones en equipos: conflictos y buenas prácticas	30 min
5.9	Patrón Repositorio y Unidad de Trabajo en aplicaciones empresariales	30 min
5.10	Logging y diagnóstico en Entity Framework Core	30 min
5.11	Testing con EF Core	30 min
5.12	Buenas prácticas y anti-patronos en persistencia empresarial	30 min

Criterio del módulo: las afirmaciones técnicas se apoyan en comportamiento ejecutable de AceriaData. No se publican timings prefijados como conclusiones ni se presentan reglas de diseño dependientes del contexto como leyes universales.

Punto 5.1 — Concurrencia optimista: concepto y necesidad

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL, sin experiencia previa en ORMs ni en Entity Framework Core.

Proyecto: Este punto continúa el proyecto acumulativo AceriaData y estudia concurrencia optimista: concepto y necesidad sobre .NET 8, Entity Framework Core 8 y SQL Server LocalDB.

Objetivos de aprendizaje

- Comprender qué problema resuelve la concurrencia optimista y por qué aparece en aplicaciones con varios usuarios.
- Distinguir una actualización perdida real de dos cambios independientes sobre propiedades distintas.
- Observar el SQL que EF Core 8 ejecuta cuando todavía no existe un token de concurrencia.
- Entender por qué el tracking por propiedad influye en el resultado de dos actualizaciones concurrentes.
- Comparar el enfoque optimista con técnicas de bloqueo sin convertir la comparación en una regla universal.
- Preparar el modelo y el razonamiento necesarios para introducir tokens en el punto 5.2.

Consideraciones técnicas en EF Core 8

La demostración distingue explícitamente el conflicto sobre la misma propiedad de cambios independientes sobre propiedades distintas. El SQL usado como evidencia es el observado durante la ejecución.

Desarrollo teórico

Qué significa concurrencia en AceriaData

La concurrencia aparece cuando dos unidades de trabajo leen el mismo estado y toman decisiones antes de que ambas hayan terminado. En AceriaData el ejemplo se realiza con dos scopes de DI y, por tanto, con dos DbContext independientes. Esa independencia es esencial: dos referencias a la misma instancia de DbContext no reproducen el problema real de dos usuarios o dos peticiones.

El punto no comienza configurando un token. Primero observa qué hace EF Core 8 sin él. El interceptor captura los comandos ejecutados y permite relacionar el resultado final con los UPDATE que realmente llegaron a SQL Server.

Actualización perdida sobre la misma propiedad

Una actualización perdida se demuestra de forma fiable cuando los dos actores modifican la misma propiedad a partir de versiones antiguas de la fila. A guarda Cliente=A y, después, B guarda Cliente=B con una entidad que había sido cargada antes de la actualización de A. Sin token, el UPDATE de B puede afectar una fila y EF Core no tiene una señal para considerar el guardado conflictivo.

El resultado correcto de la demostración es que el valor final coincida con B y que el cambio de A ya no esté presente. La evidencia es funcional y se acompaña del SQL observado.

Cambios concurrentes en propiedades distintas

El seguimiento de cambios de EF Core es por propiedad. Si A modifica Cliente y B modifica Estado, cada SaveChanges genera normalmente un UPDATE con la propiedad que se marcó como modificada. Por eso no es correcto afirmar que el segundo guardado siempre reescribe toda la entidad.

Este escenario separa dos ideas: que EF Core no detecte una modificación externa de la fila y que necesariamente se pierdan todos los cambios. La primera es cierta sin token; la segunda depende de qué columnas actualiza cada operación.

Optimista frente a bloqueo

La concurrencia optimista evita mantener bloqueos largos mientras el usuario piensa o trabaja. Se basa en comprobar, en el momento de escribir, que la versión esperada sigue siendo válida. El bloqueo pesimista usa mecanismos de base de datos para coordinar accesos concurrentes antes de la escritura.

No existe una regla general según la cual una estrategia sea universalmente más segura o más rápida. La frecuencia de conflictos, duración de la operación, coste de reintento, nivel de contención y semántica de negocio determinan la elección.

DbUpdateConcurrencyException

EF Core lanza DbUpdateConcurrencyException cuando una operación que esperaba afectar una fila por sus condiciones de concurrencia afecta cero filas. En 5.1 todavía no existe la condición adicional necesaria; por eso el objetivo del punto es mostrar la ausencia de detección.

El punto 5.2 añadirá rowversion y un token de propiedad. A partir de ahí el WHERE contendrá información de la versión original y un UPDATE que no encuentre coincidencia se convertirá en conflicto observable.

Evidencia que debe conservarse

La demostración se considera correcta cuando parte de un dataset conocido, usa contextos independientes, ejecuta dos escrituras y verifica el estado final con un tercer contexto AsNoTracking. Esa última lectura evita confundir la caché de primer nivel de un contexto con el estado real de la base.

El SQL mostrado en el manual debe provenir de ToQueryString cuando corresponde o del interceptor/logging cuando se habla de comandos ejecutados. No se presentan sentencias hipotéticas como si hubieran sido capturadas.

Ejemplo ejecutable del concepto

```
using var scopeA = _scopeFactory.CreateScope();
using var scopeB = _scopeFactory.CreateScope();
var contextA = scopeA.ServiceProvider.GetRequiredService<AceriaDbContext>();
```

CSHARP

```
var contextB = scopeB.ServiceProvider.GetRequiredService<AceriaDbContext>();
var ordenA = contextA.OrdenesFabricacion.Single(o => o.NumeroOrden == NumeroOrden);
var ordenB = contextB.OrdenesFabricacion.Single(o => o.NumeroOrden == NumeroOrden);
ordenA.Cliente = "Cliente actualizado por A";
contextA.SaveChanges();
ordenB.Cliente = "Cliente actualizado por B";
contextB.SaveChanges();
```

El ejemplo refleja la misma línea de implementación que usa el estado validado del proyecto. La práctica trabaja con el archivo real y comprueba el resultado en SQL Server LocalDB.

Profundización y contexto de uso

Profundización 1

Un formulario de edición largo es un caso clásico para concurrencia optimista. El usuario puede abrir una orden, dedicar varios minutos a revisarla y guardar después. Mantener un bloqueo de base de datos durante toda esa interacción sería costoso y frágil. Un token permite trabajar sin ese bloqueo y verificar la versión en el momento de persistir.

Profundización 2

Un proceso automático de corta duración puede tener necesidades distintas. Si varias instancias compiten por la misma fila para adjudicarse trabajo, puede ser preferible diseñar una operación atómica de base de datos o un mecanismo de cola en lugar de usar el mismo patrón que un formulario humano.

Profundización 3

También importa distinguir conflicto técnico de conflicto de negocio. Dos usuarios pueden cambiar columnas diferentes y no producir una pérdida física, pero el resultado combinado puede violar una regla del dominio. La capa de aplicación sigue siendo responsable de las invariantes.

Profundización 4

Cuando un usuario afirma que EF Core ha pisado sus datos, conviene reconstruir el orden exacto: qué contexto cargó primero, qué propiedades estaban modificadas, qué SaveChanges se ejecutó antes y qué SQL llegó al proveedor. Sin esa secuencia es fácil atribuir a concurrencia un problema que procede de estado desconectado o de una actualización demasiado amplia.

Mini caso de aplicación

Una orden abierta por dos supervisores ilustra la diferencia: si ambos cambian Cliente, hay competición directa y sin token puede prevalecer el último guardado; si uno cambia Cliente y otro Estado, el seguimiento por propiedad puede conservar ambos valores.

Patrón de diagnóstico y preguntas de revisión

La técnica se revisa partiendo de una hipótesis concreta, una evidencia observable y una comprobación del estado final. Antes de convertir una recomendación en regla general conviene responder:

- ¿Los actores usan contextos realmente independientes?
- ¿Ambos leen antes de que se produzca la primera escritura?
- ¿Compiten por la misma propiedad o por propiedades diferentes?
- ¿La comprobación final lee desde SQL Server y no desde una entidad ya rastreada?
- ¿La aplicación necesita detectar cualquier cambio de fila o solo cambios concretos?
- ¿Un conflicto puede resolverse automáticamente o debe intervenir el usuario?

Decisiones y trade-offs

Opción	Qué aporta	Trade-off
--------	------------	-----------

| Sin token | No detecta cambios externos de la fila | Sencillo, pero puede existir last-write-wins sobre la misma propiedad. |

| Optimista | Detecta en escritura mediante token | Requiere política de resolución. |

| Bloqueo | Coordina antes de escribir | Aumenta bloqueo y acoplamiento al proveedor. |

Errores frecuentes

- Usar el mismo DbContext para simular dos usuarios.
- Afirmar que dos propiedades distintas provocan necesariamente una actualización perdida.
- Confundir el estado rastreado con el estado real de la base.
- Escribir SQL esperado a mano y presentarlo como capturado.

Diagnóstico razonado del escenario concurrente

Reconstruir la secuencia antes de interpretar el resultado

Cuando aparece una actualización perdida, el primer trabajo no consiste en elegir una API, sino en reconstruir la cronología. Hay que anotar qué contexto leyó primero, qué valores observó cada actor, qué propiedades cambió cada uno y en qué orden se ejecutaron los `SaveChanges`. Dos operaciones que parecen concurrentes desde la interfaz pueden no estar modificando la misma información, y dos modificaciones de propiedades diferentes no producen necesariamente el mismo efecto que dos escrituras sobre la misma propiedad.

En AceriaData la verificación final se hace con un tercer DbContext y `AsNoTracking`. Este detalle evita una confusión frecuente: consultar de nuevo desde uno de los contextos que ya rastrea la entidad puede devolver el objeto que está en su `ChangeTracker`, no una observación independiente del estado persistido. Separar lectura inicial, escrituras y lectura de verificación vuelve reproducible la demostración.

Mirar propiedades modificadas y no solo entidades

EF Core mantiene estado a nivel de propiedad. En un contexto con tracking normal, cambiar `Cliente` no equivale a marcar todas las columnas como modificadas. Antes de concluir que el segundo usuario "machaca la fila", conviene inspeccionar `Entry(entity).Properties` y observar qué propiedades tienen `IsModified=true`. Este matiz explica por qué dos usuarios pueden conservar cambios distintos aun cuando EF Core no disponga todavía de un token que detecte que la fila cambió entre lectura y escritura.

El comportamiento puede ser diferente en escenarios desconectados si una aplicación adjunta un objeto y marca la entidad completa como modificada. Por eso una conclusión sobre concurrencia debe describir también cómo se materializa y cómo se vuelve a adjuntar el estado. La semántica de `Update` sobre un grafo desconectado no debe confundirse con la detección automática de cambios sobre una entidad cargada y rastreada.

Evidencia mínima para afirmar que hubo una pérdida

Una demostración sólida conserva cuatro piezas de evidencia: valor inicial, intención de A, intención de B y valor final persistido. A eso se añade el SQL ejecutado. Si los dos actores cambian `Cliente`, A guarda primero y B guarda después con una copia antigua, el resultado final de B permite hablar de actualización perdida. Si A cambia `Cliente` y B cambia `Estado`, el resultado debe analizarse propiedad por propiedad.

No hace falta inventar porcentajes de pérdida ni tiempos para demostrar el problema. La necesidad del control de concurrencia se deduce del resultado funcional: el sistema ha aceptado una escritura basada en una versión que ya no era la actual y no ha advertido al consumidor.

Cuándo la concurrencia optimista encaja bien

La estrategia optimista es especialmente natural cuando las colisiones son posibles pero no dominan la carga. El dato se lee sin mantener un bloqueo durante todo el tiempo de interacción y, en la escritura, se comprueba que sigue siendo la versión esperada. Si hay conflicto, la aplicación decide si informa, recarga, fusiona o reintenta.

Si la probabilidad de conflicto es muy alta o la operación no puede repetirse con seguridad, pueden ser necesarios otros mecanismos de coordinación. El punto importante es que el token no es una receta de rendimiento: es una herramienta para preservar una determinada semántica de edición concurrente.

Lista de comprobación conceptual

- Usar dos unidades de trabajo realmente independientes.
- Cargar ambos estados antes de la primera escritura cuando se quiere reproducir una copia obsoleta.
- Registrar qué propiedades modifica cada actor.
- Capturar los comandos ejecutados, no SQL inventado.
- Verificar el estado final desde un contexto independiente.
- Separar "la fila cambió" de "se perdió necesariamente todo el cambio anterior".

Escenario de decisión: edición de una orden en dos pantallas

Supongamos que dos operadores abren la misma orden de fabricación. Ambos leen `Cliente=Constructora del Norte` y `Estado=Pendiente`. El operador A corrige el cliente y guarda. El operador B, que todavía conserva la versión anterior, cambia también el cliente y guarda después. Sin token, el segundo `UPDATE` puede completar correctamente porque la clave primaria sigue identificando una fila existente. El sistema no dispone de una condición que diga que la escritura se basa en una versión antigua.

Ahora cambiamos solo una variable: B modifica `Estado` en lugar de `Cliente`. Con tracking normal, EF Core puede emitir un `UPDATE` limitado a `Estado`, por lo que la corrección de `Cliente` realizada por A se conserva. El sistema sigue sin detectar que la fila había cambiado, pero no se produce la misma pérdida. Este contraste es útil para no enseñar una simplificación falsa del tipo "dos contextos siempre sobrescriben toda la fila".

Para decidir si hace falta un token, la pregunta de negocio es: ¿debe B ser advertido de cualquier cambio realizado desde su lectura, o solo de cambios que invalidan la decisión que está tomando? El punto 5.1 no impone todavía la respuesta; construye el problema que 5.2 hará detectable.

Anclaje en AceriaData

El concepto se implementa en `M05/PROYECTO0/5.1`. La carpeta contiene su propia `AceriaData.sln` y continúa directamente desde el punto anterior. La práctica restaura, compila y ejecuta ese estado completo; cuando el punto no cambia el modelo, también se comprueba que no existan cambios de modelo pendientes.

Resumen de la teoría

- Comprender qué problema resuelve la concurrencia optimista y por qué aparece en aplicaciones con varios usuarios.
- Distinguir una actualización perdida real de dos cambios independientes sobre propiedades distintas.
- Observar el SQL que EF Core 8 ejecuta cuando todavía no existe un token de concurrencia.
- Entender por qué el tracking por propiedad influye en el resultado de dos actualizaciones concurrentes.
- Comparar el enfoque optimista con técnicas de bloqueo sin convertir la comparación en una regla universal.
- Preparar el modelo y el razonamiento necesarios para introducir tokens en el punto 5.2.

Punto 5.2 — Configuración de tokens de concurrencia

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL, sin experiencia previa en ORMs ni en Entity Framework Core.

Proyecto: Este punto continúa el proyecto acumulativo `AceriaData` y estudia configuración de tokens de concurrencia sobre `.NET 8`, `Entity Framework Core 8` y `SQL Server LocalDB`.

Objetivos de aprendizaje

- Comprender qué convierte una propiedad en token de concurrencia en EF Core 8.
- Configurar `rowversion` de SQL Server explícitamente con `IsRowVersion()`.
- Configurar un token de propiedad con `IsConcurrencyToken()`.
- Interpretar el `WHERE` del `UPDATE` cuando existe un token.
- Diferenciar `rowversion` de un token gestionado por la aplicación.
- Comprobar que `rowversion` no crea por sí mismo un índice en SQL Server.
- Generar y aplicar la migración real del módulo.

Consideraciones técnicas en EF Core 8

`rowversion` se configura explícitamente con `IsRowVersion()`; un token de propiedad con `IsConcurrencyToken()`. La práctica no presupone índices automáticos.

Desarrollo teórico

Qué es un token de concurrencia

Un token de concurrencia es una propiedad cuyo valor original forma parte de la condición de actualización o eliminación. EF Core guarda el valor original cuando materializa la entidad y lo usa al construir la operación de escritura.

Si la fila cambió y el token ya no coincide, el UPDATE no encuentra la combinación de clave y versión original. El proveedor devuelve cero filas afectadas y EF Core transforma ese hecho en `DbUpdateConcurrencyException`.

rowversion en SQL Server

rowversion es un tipo específico de SQL Server que cambia automáticamente cuando la fila se modifica. En el modelo .NET se representa habitualmente como `byte[]` y en `AceriaData` se configura de forma explícita con `IsRowVersion()`. Esa configuración marca la propiedad como token y como valor generado por la base.

El nombre `RowVersion` por sí solo no constituye una convención suficiente para enseñar el comportamiento. La práctica muestra la configuración y la migración que crea la columna real.

Token de propiedad

No todos los conflictos requieren rowversion. `IsConcurrencyToken()` permite usar el valor original de una propiedad de negocio. En `AceriaData`, `DetalleOrden.EstadoDetalle` se usa para demostrar que una propiedad normal puede participar en la condición de concurrencia.

El trade-off es distinto: rowversion cambia ante cualquier modificación de la fila; un token de propiedad solo cambia si cambia esa propiedad. El ámbito de detección debe corresponder con la semántica del negocio.

SQL del UPDATE

Con un token configurado, el SQL de escritura no filtra únicamente por la clave. También compara el valor original del token. En el caso de rowversion, el nuevo valor se obtiene del proveedor después de una actualización correcta.

La práctica captura comandos reales. No se fija una forma textual exacta que deba coincidir en todas las versiones del proveedor.

rowversion no implica índice

Un token de concurrencia no es un índice. SQL Server no crea un índice sobre una columna simplemente porque sea rowversion o porque EF Core la use en el WHERE de concurrencia.

`AceriaData` consulta `sys.indexes`, `sys.index_columns` y `sys.columns` para comprobar la realidad del esquema. Si una carga concreta necesita un índice, debe diseñarse y justificarse por su patrón de acceso.

Migración M5_5_2_ConcurrencyTokens

5.2 es el punto del módulo que cambia el modelo oficial. Por eso genera una migración real y actualiza el snapshot. Los puntos posteriores conservan esa migración como última migración oficial mientras no cambien el modelo.

La comprobación `has-pending-model-changes` de los puntos posteriores verifica que código, snapshot y migraciones siguen alineados.

Ejemplo ejecutable del concepto

```
b.Property(x => x.RowVersion).IsRowVersion();
b.Property(x => x.EstadoDetalle)
    .IsRequired()
    .HasMaxLength(50)
    .HasDefaultValue("Pendiente")
    .IsConcurrencyToken();
```

CSHARP

El ejemplo refleja la misma línea de implementación que usa el estado validado del proyecto. La práctica trabaja con el archivo real y comprueba el resultado en SQL Server LocalDB.

Profundización y contexto de uso

Profundización 1

Un token de fila como `rowversion` es apropiado cuando cualquier modificación concurrente debe invalidar una edición antigua. Un token de propiedad expresa una regla más estrecha. El caso no se resuelve preguntando qué token es mejor, sino qué modificación debe considerarse incompatible.

Profundización 2

Un Guid, un contador o una marca gestionada por la aplicación puede actuar como token si se configura con `IsConcurrencyToken()`. La aplicación debe renovarlo cuando corresponda. Esta opción puede ser portable entre proveedores, pero introduce una responsabilidad adicional.

Profundización 3

Añadir un token a una tabla existente requiere pensar en datos ya almacenados y compatibilidad durante el despliegue. En SQL Server, la nueva columna `rowversion` obtiene valores generados por el motor. En una aplicación distribuida conviene coordinar el momento en que código antiguo y nuevo conviven.

Profundización 4

Si la aplicación no lanza `DbUpdateConcurrencyException` cuando se espera, el primer paso es inspeccionar la metadata y confirmar que EF Core reconoce la propiedad como token. Después se revisan migración y esquema real.

Mini caso de aplicación

Supón una orden cuya edición debe invalidarse ante cualquier cambio. `rowversion` encaja bien. Si solo interesa impedir que dos usuarios cambien simultáneamente `EstadoDetalle`, un token sobre esa propiedad expresa una regla más estrecha.

Patrón de diagnóstico y preguntas de revisión

La técnica se revisa partiendo de una hipótesis concreta, una evidencia observable y una comprobación del estado final. Antes de convertir una recomendación en regla general conviene responder:

- ¿`RowVersion` está configurado con `IsRowVersion()` en el modelo efectivo?
- ¿La migración aplicada contiene la columna esperada?
- ¿El valor cambia después de una escritura correcta?
- ¿Se está usando SQL Server cuando se afirma comportamiento de `rowversion`?
- ¿Existe un índice porque el diseño lo requiere o se asumió que aparecería automáticamente?
- ¿El token detecta exactamente el tipo de conflicto que importa al negocio?

Decisiones y trade-offs

Opción	Qué aporta	Trade-off
--------	------------	-----------

`rowversion`	Cambia ante cualquier UPDATE de la fila	Muy cómodo en SQL Server; específico del proveedor.
Token de propiedad	Solo protege el valor elegido	Más selectivo; exige decidir qué propiedad representa el conflicto.
Token de aplicación	La app genera el valor	Portable; la aplicación debe actualizarlo correctamente.

Errores frecuentes

- Suponer que una propiedad llamada `RowVersion` se configura sola.
- Suponer que `rowversion` crea automáticamente un índice.
- Probar `rowversion` de SQL Server con SQLite.
- Modificar el modelo sin generar o actualizar la migración correspondiente.

Diagnóstico y diseño de tokens

Verificar que EF Core considera realmente la propiedad un token

Cuando se espera una `DbUpdateConcurrencyException` y no aparece, la comprobación debe comenzar en el modelo de EF Core. La propiedad tiene que estar marcada como token de concurrencia y, en el caso de `rowversion`, además debe estar configurada para generación en inserción y actualización. `IsRowVersion()` expresa precisamente esa combinación para SQL Server. Ver solo una propiedad `byte[]` en la clase no demuestra que el mecanismo esté activo.

Después se comprueba la migración y el esquema físico. El modelo puede ser correcto mientras una base concreta todavía no ha recibido la migración. En AceriaData la migración `M5_5_2_ConcurrencyTokens` es el punto en el que el cambio de modelo se convierte en columnas reales. La prueba completa recorre las tres capas: metadatos de EF, historial de migraciones y comportamiento del proveedor.

Interpretar el UPDATE de concurrencia

La señal decisiva está en el `WHERE`. Con un token, la actualización no se dirige solo por clave primaria; también compara el valor original del token. Si otra escritura cambió el `rowversion`, la segunda sentencia no encuentra la combinación esperada. EF Core interpreta que cero filas afectadas contradice la expectativa de actualizar una entidad existente y genera la excepción de concurrencia.

Este comportamiento también explica por qué asignar manualmente el valor actual a la propiedad `RowVersion` no es el mecanismo normal. En SQL Server el motor genera el nuevo valor; EF Core recoge el valor resultante después de una escritura satisfactoria y lo usa en operaciones posteriores.

Elegir la granularidad del token

Un `rowversion` de fila detecta cualquier actualización que cambie esa fila. Es una estrategia sencilla y fuerte, pero también puede declarar conflicto cuando dos cambios independientes podrían haberse fusionado según reglas de negocio. Un token de propiedad permite una granularidad distinta: solo determinadas modificaciones forman parte de la condición de concurrencia.

La elección no puede reducirse a "rowversion es mejor". Una pantalla de edición de una orden completa puede preferir detectar cualquier alteración. Un proceso que modifica campos independientes podría necesitar una política más específica. Lo importante es que el token represente qué cambios vuelven obsoleta la decisión del usuario.

Índices y rowversion

`rowversion` no equivale a índice. La columna puede participar en un predicado de actualización y seguir sin tener un índice dedicado. Crear un índice es otra decisión de diseño físico y debe justificarse por las consultas y escrituras reales. Añadirlo automáticamente por el mero hecho de ser token puede aumentar coste de mantenimiento sin aportar valor al acceso predominante.

AceriaData comprueba explícitamente `sys.indexes`, `sys.index_columns` y `sys.columns` para separar ambos conceptos. Esta comprobación evita enseñar una relación causal que SQL Server no establece por sí mismo.

Lista de comprobación conceptual

- Confirmar el token en los metadatos de EF Core.
- Confirmar la migración que crea o configura la columna.
- Confirmar el esquema de la base que realmente se está usando.
- Capturar el `UPDATE` y localizar clave y token en el `WHERE`.
- Verificar que el primer guardado cambia el token generado por SQL Server.
- No convertir el token en una recomendación automática de indexación.

Escenario de decisión: token de fila o token de propiedad

Una pantalla de mantenimiento edita varios datos de la orden y el responsable quiere saber si cualquier otro proceso tocó esa fila desde que se abrió. `rowversion` encaja bien porque cambia cuando SQL Server actualiza la fila. La copia leída por el cliente conserva el valor original y el `UPDATE` posterior puede exigir que ese valor siga siendo el actual.

En otro proceso, solo `EstadoDetalle` debe participar en la decisión concurrente. El laboratorio muestra `IsConcurrencyToken()` sobre esa propiedad. Si el estado cambia entre lectura y escritura, la condición deja de coincidir. El ejemplo permite comparar dos granularidades sin concluir que una sea universalmente superior.

Si empiezan a aparecer conflictos inesperados, la investigación revisa qué propiedades forman parte de la condición y qué operaciones las modifican. Si no aparecen conflictos cuando deberían, se revisan metadata, migración, esquema físico y SQL ejecutado. Así el diagnóstico sigue una cadena comprobable en lugar de empezar cambiando código al azar.

Anclaje en AceriaData

El concepto se implementa en `M05/PROYECTO0/5.2`. La carpeta contiene su propia `AceriaData.sln` y continúa directamente desde el punto anterior. La práctica restaura, compila y ejecuta ese estado completo; cuando el punto no cambia el modelo, también se comprueba que no existan cambios de modelo pendientes.

Resumen de la teoría

- Comprender qué convierte una propiedad en token de concurrencia en EF Core 8.
- Configurar rowversion de SQL Server explícitamente con `IsRowVersion()`.
- Configurar un token de propiedad con `IsConcurrencyToken()`.
- Interpretar el WHERE del UPDATE cuando existe un token.
- Diferenciar rowversion de un token gestionado por la aplicación.
- Comprobar que rowversion no crea por sí mismo un índice en SQL Server.

Punto 5.3 — Resolución de conflictos de concurrencia

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL, sin experiencia previa en ORMs ni en Entity Framework Core.

Proyecto: Este punto continúa el proyecto acumulativo `AceriaData` y estudia resolución de conflictos de concurrencia sobre .NET 8, Entity Framework Core 8 y SQL Server LocalDB.

Objetivos de aprendizaje

- Capturar `DbUpdateConcurrencyException` y obtener los valores original, actual y de base de datos.
- Implementar políticas cliente-gana, base-gana y merge consciente.
- Usar `GetDatabaseValues`, `OriginalValues`, `CurrentValues` y `Reload`.
- Tratar correctamente el caso en el que la fila ha sido eliminada.
- Aplicar reintentos acotados para evitar bucles infinitos.
- Comparar políticas por operaciones y roundtrips observables, no por tiempos prefijados.

Consideraciones técnicas en EF Core 8

Las políticas se implementan con `GetDatabaseValues`, `OriginalValues`, `CurrentValues` y `Reload`, y los reintentos tienen límite.

Desarrollo teórico

Los tres conjuntos de valores

Cuando aparece `DbUpdateConcurrencyException` conviene distinguir `OriginalValues`, `CurrentValues` y los valores actuales de base de datos obtenidos con `GetDatabaseValues()`. Los originales son la referencia que se usó para el control de concurrencia; los actuales contienen la intención local; los de base reflejan lo que otro actor ya confirmó.

Una resolución de conflicto es una política sobre esos tres conjuntos. No es un simple catch que ignora la excepción.

Cliente gana conscientemente

La estrategia cliente-gana conserva los valores locales, pero primero actualiza `OriginalValues` con la versión que existe en base. Así el siguiente intento usa una nueva línea base de concurrencia.

Esto puede sobrescribir trabajo confirmado por otro usuario. Por eso debe ser una decisión de negocio explícita y, en interfaces humanas, suele requerir información suficiente para que el usuario entienda qué está sustituyendo.

Base de datos gana

Base-gana descarta la intención local y recarga la entidad. `Reload()` sustituye valores actuales y originales por el estado de la base.

Es apropiado cuando el servidor o el proceso que escribió antes tiene autoridad, o cuando el usuario prefiere volver a cargar y reevaluar su cambio.

Merge por propiedad

Una resolución personalizada permite elegir por propiedad. El laboratorio conserva Cliente de B y Estado de la base tras el cambio de A. Para poder reintentar, OriginalValues se actualiza con la versión de base; luego CurrentValues se ajusta según la política.

El merge solo es seguro si la aplicación conoce la semántica de las propiedades. Combinar ciegamente campos puede producir un estado válido técnicamente pero incoherente para el negocio.

Fila eliminada

GetDatabaseValues() devuelve null cuando la fila ya no existe. Ese caso no debe tratarse como si hubiera una versión más nueva que se pudiera fusionar.

AceriaData reconoce la eliminación y desacopla la entrada. El comportamiento final puede ser notificar, recrear o cancelar, pero la decisión debe ser explícita.

Reintentos acotados

Un reintento puede resolver un conflicto transitorio, pero un while infinito bajo contención convierte la resolución en un problema de disponibilidad. El laboratorio cuenta los intentos y fija un máximo.

La evidencia útil incluye el número de intentos y los comandos adicionales. Los tiempos de una única máquina no se convierten en una clasificación universal de estrategias.

Ejemplo ejecutable del concepto

```
catch (DbUpdateConcurrencyException ex)
{
    var entry = ex.Entries.Single();
    var db = entry.GetDatabaseValues()
        ?? throw new InvalidOperationException("La fila ya no existe.");
    entry.OriginalValues.SetValues(db);
    entry.CurrentValues[nameof(OrdenFabricacion.Estado)] = db[nameof(OrdenFabricacion.Estado)];
    contextB.SaveChanges();
}
```

CSHARP

El ejemplo refleja la misma línea de implementación que usa el estado validado del proyecto. La práctica trabaja con el archivo real y comprueba el resultado en SQL Server LocalDB.

Profundización y contexto de uso

Profundización 1

La política de resolución debe vivir cerca del caso de uso que conoce la intención del usuario. Un repositorio puede exponer datos necesarios, pero no debería decidir arbitrariamente que el cliente gana en todos los escenarios.

Profundización 2

Cuando la interfaz de usuario participa, es útil presentar valores originales, valores que el usuario intentaba guardar y valores actuales de la base. Esta triple comparación permite explicar el conflicto sin convertirlo en un mensaje genérico de error.

Profundización 3

Un merge por propiedad no debe limitarse a combinar valores. Después de construir el estado candidato es necesario volver a validar invariantes del agregado. Dos cambios individualmente válidos pueden producir juntos un estado imposible.

Profundización 4

Los reintentos automáticos son adecuados cuando la política está completamente definida y la operación es segura de repetir. Si el conflicto cambia el significado de la acción, ocultarlo tras reintentos puede sorprender al usuario.

Mini caso de aplicación

Imagina que A cambia Cliente y B cambia Cliente más Estado. Con cliente gana, B puede reintentar; con base gana, B descarta; con un merge, el caso de uso puede conservar Cliente de B y Estado de A. Solo el dominio decide cuál es correcta.

Patrón de diagnóstico y preguntas de revisión

La técnica se revisa partiendo de una hipótesis concreta, una evidencia observable y una comprobación del estado final. Antes de convertir una recomendación en regla general conviene responder:

- ¿Se distinguen CurrentValues, OriginalValues y valores de base?
- ¿Se trata de forma separada la fila eliminada?
- ¿El reintento tiene un límite?
- ¿Después del merge se vuelven a comprobar invariantes?
- ¿La política puede explicar al usuario qué datos se descartaron?
- ¿La operación es segura de repetir si aparece un segundo conflicto?

Decisiones y trade-offs

Opción	Qué aporta	Trade-off
--------	------------	-----------

| Cliente gana | Conserva intención local | Puede sobrescribir cambios ajenos. |

| Base gana | Descarta intención local | Obliga a reintentar edición si el usuario aún quiere cambiar. |

| Merge | Combina por propiedad | Necesita reglas de dominio. |

| Notificar | No decide automáticamente | Más interacción, pero hace visible el conflicto. |

Errores frecuentes

- Capturar DbUpdateConcurrencyException y volver a SaveChanges sin actualizar la línea base.
- Reintentar sin límite.
- Suponer que GetDatabaseValues nunca devuelve null.
- Hacer merge sin reglas de dominio.

Diseñar una política de resolución de conflictos

Un conflicto detectado no decide quién tiene razón

`DbUpdateConcurrencyException` aporta evidencia de que la versión leída ya no coincide con la base, pero no contiene una política de negocio. La aplicación tiene que decidir qué significa el conflicto para el caso de uso. Por eso `AceriaData` demuestra varias estrategias en lugar de esconder la excepción detrás de un reintento genérico.

`OriginalValues` representa la referencia con la que se intentó escribir; `CurrentValues` representa la intención local; `GetDatabaseValues()` recupera el estado que existe ahora en la base. Esas tres vistas permiten construir una comparación explícita y, si procede, una fusión propiedad por propiedad.

Cliente gana con conocimiento del conflicto

"Cliente gana" no significa ignorar la concurrencia. La aplicación detecta el conflicto, lee la versión actual de la base, actualiza los valores originales con esa versión y vuelve a intentar conscientemente la intención local. El segundo intento ya se basa en un token actual. Esta política sobrescribe determinados valores por decisión explícita, no porque EF Core haya dejado de comprobar el token.

Debe existir un límite de intentos. Si cada reintento vuelve a entrar en conflicto por alta contención, un bucle infinito puede consumir recursos y ocultar que el dato requiere intervención o una estrategia diferente.

Base de datos gana y notificación

`Reload()` es apropiado cuando la decisión es descartar el cambio local y aceptar el estado persistido. Otra posibilidad es no tocar el estado todavía y devolver al usuario las diferencias. Una interfaz puede mostrar qué valor leyó, qué valor intentó guardar y qué valor existe en base, permitiendo una elección informada.

Estas dos políticas tienen costes de experiencia de usuario distintos. "Base gana" es simple pero puede descartar trabajo; notificar conserva la posibilidad de decidir, pero exige soporte en la aplicación y una representación comprensible de las diferencias.

Merge personalizado

La resolución personalizada expresa reglas de dominio. Por ejemplo, el cliente local puede prevalecer en `Cliente` y la base en `Estado`. La implementación necesita actualizar `OriginalValues` al estado recuperado, ajustar `CurrentValues` según la política y guardar de nuevo. El resultado final se verifica después de la escritura para demostrar que cada propiedad siguió la regla acordada.

No debe asumirse que todas las propiedades son fusionables. Totales, secuencias, estados de workflow o datos derivados pueden necesitar invariantes adicionales. El merge es código de negocio y merece pruebas propias.

Fila eliminada

`GetDatabaseValues()` puede devolver `null`. Ese caso no es un conflicto de valores sino la desaparición de la fila. Reintentar el mismo UPDATE no puede resolverlo. `AceriaData` reconoce la eliminación y desacopla la entrada según la política del laboratorio. En una aplicación real podría informarse al usuario, recrear el recurso si el dominio lo permite o cancelar la operación.

Lista de comprobación conceptual

- Capturar la excepción en la frontera que puede tomar una decisión de negocio.
- Obtener original, actual y base de datos antes de sobrescribir información.
- Definir por escrito qué campos prevalecen en una fusión.
- Limitar los reintentos.
- Tratar `GetDatabaseValues()==null` como caso propio.
- Verificar el estado final, no solo la ausencia de excepción.

Escenario de decisión: qué mostrar al usuario después del conflicto

Un operador cambia el cliente de una orden mientras otro cambia el mismo dato y guarda primero. La aplicación detecta el conflicto. Una política "base de datos gana" puede recargar y presentar el nuevo valor; una política "cliente gana" puede reintentar conscientemente la intención local; una política de notificación puede detenerse y mostrar las tres versiones relevantes.

La decisión cambia cuando las propiedades representan significados distintos. Si A cambia `Estado` porque el proceso físico ya avanzó y B corrige el nombre del cliente, una fusión personalizada podría conservar el estado de la base y el cliente local. Ese comportamiento no puede deducirse de EF Core: debe expresarse como regla del dominio y probarse con un resultado final esperado.

El número de reintentos también es una política. Un reintento acotado puede ser razonable cuando el conflicto es transitorio; repetir sin límite transforma contención en consumo indefinido. Cuando la fila desapareció, ni siquiera existe un estado de base con el que refrescar el token, por lo que el flujo debe tratar la eliminación de manera explícita.

Anclaje en `AceriaData`

El concepto se implementa en `M05/PROYECTO/5.3`. La carpeta contiene su propia `AceriaData.sln` y continúa directamente desde el punto anterior. La práctica restaura, compila y ejecuta ese estado completo; cuando el punto no cambia el modelo, también se comprueba que no existan cambios de modelo pendientes.

Resumen de la teoría

- Capturar `DbUpdateConcurrencyException` y obtener los valores original, actual y de base de datos.
- Implementar políticas cliente-gana, base-gana y merge consciente.
- Usar `GetDatabaseValues`, `OriginalValues`, `CurrentValues` y `Reload`.
- Tratar correctamente el caso en el que la fila ha sido eliminada.
- Aplicar reintentos acotados para evitar bucles infinitos.
- Comparar políticas por operaciones y roundtrips observables, no por tiempos prefijados.

Punto 5.4 — Transacciones: `SaveChanges` y transacciones explícitas

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL, sin experiencia previa en ORMs ni en Entity Framework Core.

Proyecto: Este punto continúa el proyecto acumulativo AceriaData y estudia transacciones: savechanges y transacciones explícitas sobre .NET 8, Entity Framework Core 8 y SQL Server LocalDB.

Objetivos de aprendizaje

- Comprender la atomicidad de una llamada a SaveChanges en un proveedor relacional.
- Usar BeginTransaction, Commit y Rollback cuando varias operaciones deban formar una sola unidad.
- Crear y revertir savepoints manuales.
- Entender la interacción entre savepoints y MARS en SQL Server.
- Distinguir rollback de una operación de recuperación de negocio.
- Evitar conclusiones de rendimiento basadas en cifras fijas no reproducibles.

Consideraciones técnicas en EF Core 8

Los savepoints de SQL Server se prueban con MARS desactivado. No se asume un RELEASE SAVEPOINT ni un rollback universal ante cualquier fallo de Commit.

Desarrollo teórico

Atomicidad de SaveChanges

En un proveedor relacional, una única llamada a SaveChanges agrupa sus operaciones de forma atómica cuando necesita una transacción. El laboratorio introduce una inserción válida y otra que viola una restricción para verificar que no queda persistida la primera si la llamada completa falla.

La comprobación se hace leyendo después con otro contexto. No basta con observar la excepción: hay que verificar el estado de la base.

Transacción explícita

BeginTransaction permite agrupar varias llamadas a SaveChanges y otras operaciones en una sola unidad. Commit confirma la unidad; Rollback revierte los cambios que ya llegaron al servidor dentro de esa transacción.

Usar una transacción explícita no implica que siempre sea mejor. SaveChanges ya ofrece atomicidad para su propia unidad y mantener una transacción abierta aumenta duración de bloqueos y uso de recursos.

Savepoints

Un savepoint marca un punto dentro de una transacción. Después de confirmar una primera operación, el laboratorio crea AntesSegundaOrden, realiza una segunda escritura y vuelve al savepoint. La primera permanece dentro de la transacción y la segunda se revierte.

En SQL Server el mecanismo se corresponde con SAVE TRANSACTION y ROLLBACK TRANSACTION. No se enseña RELEASE SAVEPOINT como requisito de SQL Server.

Savepoints automáticos de EF Core

Cuando SaveChanges se ejecuta dentro de una transacción ya activa, EF Core puede crear un savepoint antes de guardar. Esto facilita dejar la transacción en un estado recuperable si falla el guardado.

Con SQL Server hay una limitación importante: EF Core no crea estos savepoints cuando MARS está habilitado. Por ello el laboratorio fija MultipleActiveResultSets=false y lo comprueba.

Commit, errores y recuperación

Un error durante Commit no debe describirse con garantías simplificadas. Dependiendo del fallo y del estado de la conexión, la aplicación puede necesitar determinar si el resultado es conocido o incierto y aplicar su estrategia de resiliencia.

Rollback es una operación transaccional, no un plan completo de recuperación de negocio. Cuando ya existen efectos externos o cambios destructivos, la recuperación puede exigir compensación o restauración.

Medir transacciones

Comparar muchas llamadas independientes con una operación agrupada puede ser útil como experimento local, pero el resultado depende de batching, latencia, hardware y configuración. El curso evita publicar milisegundos prefijados.

Las evidencias más estables son atomicidad, comandos o roundtrips, duración de la transacción y estado final de los datos.

Ejemplo ejecutable del concepto

CSHARP

```
using var transaction = context.Database.BeginTransaction();
context.OrdenesFabricacion.Add(CrearOrden(numero1));
context.SaveChanges();
transaction.CreateSavepoint("AntesSegundaOrden");
context.OrdenesFabricacion.Add(CrearOrden(numero2));
context.SaveChanges();
transaction.RollbackToSavepoint("AntesSegundaOrden");
transaction.Commit();
```

El ejemplo refleja la misma línea de implementación que usa el estado validado del proyecto. La práctica trabaja con el archivo real y comprueba el resultado en SQL Server LocalDB.

Profundización y contexto de uso

Profundización 1

Una transacción debe cubrir exactamente el conjunto de cambios que necesita atomicidad. Incluir llamadas remotas, cálculos largos o interacción humana aumenta bloqueo y hace más probable timeout o deadlock.

Profundización 2

Hay casos en los que una única llamada a SaveChanges basta. Otras operaciones necesitan conocer una clave generada por la primera escritura antes de preparar la segunda o mezclan EF con comandos sobre la misma conexión. Una transacción explícita permite mantener atomicidad entre esas llamadas.

Profundización 3

Los savepoints son útiles cuando una transacción tiene fases. Se puede persistir una fase, crear un punto y probar una operación posterior que podría fallar sin perder lo anterior. El código debe entender el estado del ChangeTracker después de un rollback parcial.

Profundización 4

Mantener transacciones cortas reduce exposición a contención, pero no elimina deadlocks. Una estrategia de reintento puede ser apropiada para fallos transitorios si la unidad de trabajo es repetible y se coordina con las transacciones explícitas.

Mini caso de aplicación

Una operación crea una orden, reserva material y registra un detalle. Si las tres escrituras forman una única decisión de negocio, deben confirmar juntas. Si el detalle es opcional, un savepoint puede permitir una recuperación parcial.

Patrón de diagnóstico y preguntas de revisión

La técnica se revisa partiendo de una hipótesis concreta, una evidencia observable y una comprobación del estado final. Antes de convertir una recomendación en regla general conviene responder:

- ¿Puede la operación resolverse con un solo SaveChanges?
- ¿Qué llamadas deben confirmar de forma atómica?
- ¿La transacción contiene trabajo remoto o esperas innecesarias?
- ¿MARS está desactivado cuando se dependen de savepoints de SQL Server?
- ¿El código conoce qué hacer después de un rollback parcial?
- ¿Una estrategia de reintento envolvería la unidad transaccional completa?

Decisiones y trade-offs

Opción	Qué aporta	Trade-off
--------	------------	-----------

SaveChanges	Atomicidad de su unidad	Suficiente para muchas operaciones.
Transacción explícita	Agrupa varios SaveChanges	Aumenta duración de la transacción.
Savepoint	Rollback parcial	Depende de soporte y configuración.

Errores frecuentes

- Abrir una transacción explícita para cada `SaveChanges` sin necesidad.
- Usar savepoints con MARS habilitado y asumir el mismo comportamiento.
- Presentar `RELEASE SAVEPOINT` como SQL Server obligatorio.
- Confundir rollback técnico con recuperación de negocio.

Razonamiento operativo sobre transacciones

Separar estado del `ChangeTracker` y estado de la transacción

Una entidad puede seguir apareciendo como `Added`, `Modified` o incluso `Unchanged` en memoria mientras la transacción de base se confirma o se revierte. Tras un rollback, confiar ciegamente en los objetos del contexto puede producir una visión diferente del estado persistido. Por eso las demostraciones de `AceriaData` limpian el tracker cuando procede y verifican el resultado con una lectura independiente.

La transacción protege operaciones del proveedor; el `ChangeTracker` es una estructura de la aplicación. Entender esa frontera evita suponer que `Rollback()` rebobina automáticamente cualquier efecto en objetos .NET, cachés o sistemas externos.

Atomicidad de `SaveChanges`

En un proveedor relacional, una llamada a `SaveChanges` que necesita varias sentencias puede ejecutarse dentro de una transacción para que el conjunto sea atómico. El laboratorio inserta una fila válida y otra que viola una restricción en la misma llamada y comprueba que la válida no queda persistida cuando la operación falla.

Esto no convierte varias llamadas independientes a `SaveChanges` en una sola unidad. Si el proceso exige que un primer guardado y un segundo guardado se confirmen juntos, una transacción explícita expresa esa frontera.

Commit y rollback explícitos

`BeginTransaction` da a la aplicación control sobre la unidad de confirmación. Dos llamadas a `SaveChanges` pueden ejecutarse y confirmarse al final con `Commit`. En el escenario de error, `Rollback` revierte la primera escritura que ya había llegado a SQL Server. La verificación posterior demuestra la diferencia entre haber enviado una sentencia y haber confirmado la transacción que la contiene.

El código que captura una excepción también tiene que considerar el estado real de la transacción. No es correcto convertir cualquier fallo de `Commit` en la afirmación genérica de que todo rollback está garantizado en cualquier circunstancia; las excepciones deben tratarse de acuerdo con el proveedor y el punto de fallo.

Savepoints como recuperación parcial

Un savepoint crea un punto de retorno dentro de la misma transacción. `AceriaData` confirma la primera inserción lógica, crea `AntesSegundaOrden`, realiza una segunda escritura, vuelve al savepoint y finalmente confirma. El resultado esperado es que la primera fila permanezca y la segunda no.

En SQL Server, la sintaxis física usa `SAVE TRANSACTION` y `ROLLBACK TRANSACTION`; no se debe trasladar sin más la sintaxis de otros motores. Además, EF Core puede crear savepoints automáticamente antes de `SaveChanges` cuando ya existe una transacción explícita.

MARS y savepoints

La documentación y el laboratorio mantienen `MultipleActiveResultSets=false` porque los savepoints de EF Core no son compatibles con MARS habilitado en este escenario. Si una aplicación depende de savepoints, la cadena de conexión forma parte del comportamiento que hay que revisar; no es un detalle ajeno al diseño transaccional.

Lista de comprobación conceptual

- Definir qué operaciones forman una sola unidad atómica.
- Distinguir una llamada a `SaveChanges` de una transacción que engloba varias llamadas.
- Verificar el estado persistido desde fuera del contexto que participó en el rollback.
- Revisar MARS antes de basarse en savepoints de SQL Server.
- No usar una cifra aislada de tiempo para justificar agrupar transacciones.
- Tratar los efectos externos con mecanismos distintos de la transacción local.

Escenario de decisión: una operación con tres etapas

Una operación empresarial crea una orden, registra un detalle y actualiza un estado. Si las tres modificaciones forman una única decisión de negocio, dividir las en tres `SaveChanges` sin una transacción exterior permite que las primeras queden confirmadas aunque la última falle. Una transacción explícita puede agruparlas y hacer que la confirmación ocurra solo cuando todas las etapas han terminado.

En cambio, un proceso puede querer conservar la primera etapa aunque una segunda opcional falle. Un savepoint permite volver a un punto intermedio de la misma transacción. El diseño no consiste en elegir siempre la transacción "más grande", sino en definir qué estados parciales son válidos para el negocio.

Después de un rollback, el código no debería mirar únicamente los objetos que siguen en memoria para decidir qué quedó en SQL Server. Una lectura independiente confirma el estado real. Esta separación entre tracker y base es especialmente importante en ejemplos docentes porque evita atribuir al rollback efectos sobre objetos .NET que la transacción de SQL no controla.

Anclaje en AceriaData

El concepto se implementa en `M05/PROYECTO/5.4`. La carpeta contiene su propia `AceriaData.sln` y continúa directamente desde el punto anterior. La práctica restaura, compila y ejecuta ese estado completo; cuando el punto no cambia el modelo, también se comprueba que no existan cambios de modelo pendientes.

Resumen de la teoría

- Comprender la atomicidad de una llamada a `SaveChanges` en un proveedor relacional.
- Usar `BeginTransaction`, `Commit` y `Rollback` cuando varias operaciones deban formar una sola unidad.
- Crear y revertir savepoints manuales.
- Entender la interacción entre savepoints y MARS en SQL Server.
- Distinguir rollback de una operación de recuperación de negocio.
- Evitar conclusiones de rendimiento basadas en cifras fijas no reproducibles.

Punto 5.5 — Transacciones ambientales y buenas prácticas

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL, sin experiencia previa en ORMs ni en Entity Framework Core.

Proyecto: Este punto continúa el proyecto acumulativo `AceriaData` y estudia transacciones ambientales y buenas prácticas sobre .NET 8, Entity Framework Core 8 y SQL Server LocalDB.

Objetivos de aprendizaje

- Comprender qué es `TransactionScope` y cómo fluye una transacción ambiental.
- Usar `TransactionScopeAsyncFlowOption.Enabled` en código asíncrono.
- Diferenciar `Required`, `RequiresNew` y `Suppress`.
- Observar `ReadCommitted` y `Snapshot` sin atribuirles propiedades que no tienen.
- Entender cuándo puede producirse promoción a una transacción distribuida.
- Demostrar que un recurso externo normal no se revierte automáticamente con la base de datos.

Consideraciones técnicas en EF Core 8

`TransactionScope` se prueba con flujo async y sin depender de MSDTC; `ReadCommitted`, `Snapshot` y recursos externos se describen con sus límites reales.

Desarrollo teórico

Qué es una transacción ambiental

`TransactionScope` establece una transacción accesible mediante `Transaction.Current`. Los recursos compatibles que se abren dentro del scope pueden participar sin que cada método reciba explícitamente un objeto transacción.

La comodidad tiene un coste conceptual: hay que comprender propagación, compatibilidad del proveedor y posible promoción.

Flujo asíncrono

En código async se utiliza `TransactionScopeAsyncFlowOption.Enabled` para que la transacción ambiental fluya a través de `await`. El laboratorio comprueba `Transaction.Current` antes y después de una cesión asíncrona.

Olvidar esta opción puede provocar excepciones o que el código no mantenga la transacción que el desarrollador cree tener.

Dos DbContext y promoción

AceriaData crea dos `DbContext` sobre la misma conexión SQL abierta. Así demuestra coordinación multi-contexto sin depender accidentalmente de MSDTC. Además observa `DistributedIdentifier` para saber si hubo promoción.

Abrir recursos durables adicionales puede promover una transacción local a distribuida. En .NET moderno el soporte de `System.Transactions` distribuido es específico de Windows y requiere infraestructura adicional.

Required, RequiresNew y Suppress

`Required` reutiliza el ambiente existente cuando lo hay. `RequiresNew` crea una transacción distinta. `Suppress` ejecuta el bloque sin `Transaction.Current`.

Estas opciones cambian la frontera de atomicidad. Deben elegirse de acuerdo con qué operaciones deben confirmar o abortar juntas.

ReadCommitted y Snapshot

`ReadCommitted` evita lecturas sucias; no debe confundirse con `ReadUncommitted`. `Snapshot` utiliza versionado de filas para ofrecer lecturas consistentes respecto a un snapshot de la base.

`Snapshot` puede reducir ciertos bloqueos de lectura, pero no elimina todos los bloqueos ni los conflictos de escritura. Su coste incluye almacenamiento/versionado y requisitos de configuración de la base.

Recursos externos

Una llamada HTTP, un archivo o un mensaje a un sistema que no participa en `System.Transactions` no se deshace por omitir `Complete()`. El laboratorio conserva deliberadamente un efecto externo simulado mientras la fila SQL se revierte.

La consistencia entre base de datos y mensajería suele resolverse con patrones como outbox, idempotencia o compensación; no fingiendo que `TransactionScope` controla recursos que no están enlistados.

Ejemplo ejecutable del concepto

```
using var scope = new TransactionScope(
    TransactionScopeOption.Required,
    options,
    TransactionScopeAsyncFlowOption.Enabled);
await using var connection = new SqlConnection(connectionString);
await connection.OpenAsync();
await Task.Yield();
var flujoAsync = Transaction.Current is not null;
```

CSHARP

El ejemplo refleja la misma línea de implementación que usa el estado validado del proyecto. La práctica trabaja con el archivo real y comprueba el resultado en SQL Server LocalDB.

Profundización y contexto de uso

Profundización 1

El atractivo de `TransactionScope` es que los componentes no necesitan recibir explícitamente una transacción. Esa misma característica puede ocultar que una operación participa en un ámbito ambiental. Los equipos deben establecer convenciones claras sobre dónde se crean scopes.

Profundización 2

Mientras una única conexión física participa, el proveedor puede mantener una transacción local. Incorporar otra conexión o recurso durable puede exigir coordinación distribuida. La promoción tiene requisitos operacionales diferentes y no debe descubrirse por accidente en producción.

Profundización 3

`AsyncFlowOption.Enabled` resuelve el flujo de `Transaction.Current` a través de continuaciones, pero no significa que cualquier API asíncrona o cualquier proveedor soporte transacciones ambientales de la misma manera.

Profundización 4

Si una operación guarda en SQL y después publica un mensaje, un fallo entre ambos pasos puede dejar solo uno de los efectos. Un patrón outbox guarda el mensaje pendiente en la misma transacción de base y un proceso posterior lo publica de forma idempotente.

Mini caso de aplicación

Un servicio crea una orden con un contexto y una plancha con otro. Compartir la misma conexión permite que ambos participen en la transacción ambiental del laboratorio. Abrir otra conexión independiente cambia el escenario y puede aparecer promoción.

Patrón de diagnóstico y preguntas de revisión

La técnica se revisa partiendo de una hipótesis concreta, una evidencia observable y una comprobación del estado final. Antes de convertir una recomendación en regla general conviene responder:

- ¿Se usa `AsyncFlowOption.Enabled` en flujos con `await`?
- ¿Cuántas conexiones físicas participan en el scope?
- ¿`Required`, `RequiresNew` o `Suppress` expresan realmente la intención?
- ¿Algún efecto externo se está suponiendo transaccional sin serlo?
- ¿El aislamiento elegido responde a un fenómeno concreto?
- ¿Una transacción explícita sería más clara que el ámbito ambiental?

Decisiones y trade-offs

Opción	Qué aporta	Trade-off
--------	------------	-----------

`Required`	Reutiliza la transacción ambiental	Favorece una frontera atómica común.
`RequiresNew`	Aísla una unidad nueva	Puede confirmar o abortar de forma distinta al exterior.
`Suppress`	Ejecuta sin ambiente	Útil para operaciones que no deben participar.

Errores frecuentes

- Olvidar `TransactionScopeAsyncFlowOption.Enabled`.
- Suponer que dos conexiones nunca promueven la transacción.
- Decir que `ReadCommitted` permite lecturas sucias.
- Esperar que un servicio externo se revierta automáticamente.

Diagnóstico de transacciones ambientales

Confirmar que existe una transacción ambiental

`TransactionScope` funciona mediante `Transaction.Current`. Una primera comprobación útil es observar si esa propiedad está presente dentro del scope y si sigue presente después de un `await`. En código asíncrono se usa `TransactionScopeAsyncFlowOption.Enabled`; omitirlo puede romper el flujo esperado de la transacción a través de continuaciones asíncronas.

El laboratorio no da por hecho el nivel de aislamiento ni el timeout: también los observa. Esto es importante porque los valores predeterminados pertenecen a la configuración de `System.Transactions` y no deben convertirse en "números mágicos" del manual.

Varios `DbContext` no implican obligatoriamente una transacción distribuida

`AceriaData` crea dos `DbContext` sobre una misma conexión SQL abierta. De ese modo demuestra coordinación de dos contextos sin hacer que el ejercicio dependa accidentalmente de MSDTC. El hecho de que existan dos objetos `DbContext` no determina por sí solo cuántos recursos durables participan.

Para diagnosticar una posible promoción hay que observar conexiones y `DistributedIdentifier`, además del entorno. Abrir recursos independientes puede hacer que una transacción local tenga que coordinarse de otra forma. En .NET moderno, las transacciones distribuidas tienen restricciones de plataforma y despliegue que deben considerarse antes de adoptar el patrón.

Required, RequiresNew y Suppress

`Required` reutiliza la transacción ambiental existente cuando la hay. `RequiresNew` crea una nueva frontera transaccional. `Suppress` ejecuta un bloque sin la transacción ambiental. Estas opciones permiten expresar composición, pero también pueden hacer más difícil razonar sobre el sistema si se anidan sin un objetivo claro.

Una revisión de código útil identifica cada frontera y pregunta qué ocurre si el bloque interior confirma o falla. El nombre del método no basta; hay que seguir `Transaction.Current` y el ciclo de vida de cada scope.

ReadCommitted y Snapshot

`ReadCommitted` evita lecturas sucias; no debe describirse como si permitiera leer datos no confirmados. `Snapshot` ofrece lecturas consistentes basadas en versionado de filas cuando SQL Server está configurado para ello. Eso no significa "sin bloqueos" ni elimina conflictos de escritura.

El aislamiento se elige por anomalías que el negocio puede tolerar, coste del version store, contención y comportamiento de consultas/escrituras. La comparación debe hacerse sobre una carga representativa si se quiere hablar de rendimiento.

Recursos externos

Una llamada HTTP, un fichero, un email o una cola que no participa en `System.Transactions` no se revierte porque el scope SQL haga rollback. `AceriaData` simula un efecto externo y demuestra que puede permanecer mientras la fila de base desaparece. Esta observación lleva a patrones como outbox, idempotencia del consumidor o compensaciones cuando se necesita consistencia entre sistemas.

Lista de comprobación conceptual

- Comprobar `Transaction.Current` y el flujo asíncrono.
- Contar recursos/conexiones reales antes de hablar de promoción.
- Elegir `Required`, `RequiresNew` o `Suppress` por semántica, no por costumbre.
- Definir explícitamente aislamiento y timeout cuando importan.
- No atribuir rollback automático a recursos que no participan en la transacción.
- Probar el escenario de ausencia de `Complete()` y verificar el estado final.

Escenario de decisión: dos contextos y un efecto externo

Un servicio crea una orden con un contexto y una plancha con otro dentro de un `TransactionScope`. Si ambos usan la misma conexión SQL abierta, el laboratorio puede observar una única transacción ambiental sin depender de una promoción distribuida. Al salir del scope sin `Complete`, las escrituras SQL se revierten y la verificación externa confirma que no persisten.

Añadimos ahora un efecto que no participa en `System.Transactions`: por ejemplo, registrar un evento en una colección que representa un sistema externo. El rollback de SQL no puede deshacer ese efecto. El resultado hace visible por qué una transacción ambiental no convierte HTTP, correo o una cola ordinaria en recursos transaccionales coordinados.

Cuando una arquitectura necesita consistencia entre base y mensajería, la solución debe diseñarse explícitamente. El punto no prescribe una infraestructura concreta; sí establece la pregunta correcta: qué recursos están realmente enlistados y qué ocurre con cada uno cuando la operación principal falla.

Anclaje en AceriaData

El concepto se implementa en `M05/PROYECTO0/5.5`. La carpeta contiene su propia `AceriaData.sln` y continúa directamente desde el punto anterior. La práctica restaura, compila y ejecuta ese estado completo; cuando el punto no cambia el modelo, también se comprueba que no existan cambios de modelo pendientes.

Resumen de la teoría

- Comprender qué es `TransactionScope` y cómo fluye una transacción ambiental.
- Usar `TransactionScopeAsyncFlowOption.Enabled` en código asíncrono.
- Diferenciar `Required`, `RequiresNew` y `Suppress`.

- Observar ReadCommitted y Snapshot sin atribuirles propiedades que no tienen.
- Entender cuándo puede producirse promoción a una transacción distribuida.
- Demostrar que un recurso externo normal no se revierte automáticamente con la base de datos.

Punto 5.6 — Migraciones en entornos de producción: estrategias y despliegue

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL, sin experiencia previa en ORMs ni en Entity Framework Core.

Proyecto: Este punto continúa el proyecto acumulativo AceriaData y estudia migraciones en entornos de producción: estrategias y despliegue sobre .NET 8, Entity Framework Core 8 y SQL Server LocalDB.

Objetivos de aprendizaje

- Distinguir estrategias de aplicación de migraciones en desarrollo y producción.
- Generar SQL revisable y migration bundles.
- Usar IMigrator en un laboratorio controlado.
- Conservar correctamente __EFMigrationsHistory.
- Entender el riesgo de cambiar MigrationsHistoryTable en una base ya desplegada.
- Preparar preflight, backup, verificación y reversión.
- Diferenciar downgrade de recuperación segura.

Consideraciones técnicas en EF Core 8

El despliegue prioriza artefactos controlables. AceriaData conserva __EFMigrationsHistory y no usa migración de arranque como regla general.

Desarrollo teórico

Migraciones en producción no son migraciones de desarrollo

En desarrollo es habitual ejecutar comandos interactivos. En producción, el cambio de esquema forma parte de un despliegue con revisión, permisos, observabilidad y recuperación.

Por eso el punto ordena las estrategias por contexto: SQL revisable, migration bundle, CLI en entorno controlado y migración en runtime solo cuando se aceptan conscientemente sus trade-offs.

MigrationsAssembly

La infraestructura configura explícitamente el ensamblado que contiene migraciones. Esto elimina ambigüedad cuando el startup project es distinto del proyecto que contiene AceriaDbContext y las migraciones.

La configuración se conserva en todos los puntos posteriores y el build valida que la cadena se puede descubrir desde el startup project.

__EFMigrationsHistory

EF Core registra migraciones aplicadas en __EFMigrationsHistory. Cambiar el nombre de esa tabla en una base ya existente sin mover el historial puede hacer que EF Core pierda la referencia del estado aplicado.

La personalización es válida si se diseña desde el principio o se acompaña de una operación explícita sobre el historial. AceriaData conserva el nombre heredado.

IMigrator

IMigrator expone el servicio de migración para un flujo programático controlado. El laboratorio parte de una base de demostración y aplica la cadena completa, luego comprueba aplicadas, pendientes y última migración.

Esto enseña la API sin convertir Database.Migrate() en una recomendación automática de arranque para múltiples réplicas de producción.

Scripts y bundles

dotnet ef migrations script produce SQL que puede revisarse, aprobarse y versionarse como artefacto de despliegue. dotnet ef migrations bundle genera un ejecutable que encapsula la aplicación de migraciones.

La cadena de conexión de producción no se incrusta en el repositorio ni en el bundle. Se suministra desde secretos del entorno de despliegue.

Preflight, backup y reversión

Un despliegue serio incluye comprobación de versión, espacio, permisos y estado; una copia de seguridad cuando el riesgo lo justifique; aplicación; verificación posterior; y un plan de recuperación.

Un downgrade ejecuta métodos Down y puede eliminar datos. No es equivalente a restaurar un backup ni garantiza recuperar información perdida.

Ejemplo ejecutable del concepto

```
dotnet ef migrations script --idempotent `
  --project src/AceriaData.Infrastructure `
  --startup-project src/AceriaData.Console `
  --configuration Release `
  --output deployment/artifacts/aceria-idempotent.sql

dotnet ef migrations bundle `
  --project src/AceriaData.Infrastructure `
  --startup-project src/AceriaData.Console `
  --configuration Release `
  --output deployment/artifacts/aceria-efbundle.exe
```

POWERSHELL

El ejemplo refleja la misma línea de implementación que usa el estado validado del proyecto. La práctica trabaja con el archivo real y comprueba el resultado en SQL Server LocalDB.

Profundización y contexto de uso

Profundización 1

El equipo que desarrolla la migración conoce la intención del modelo; el sistema de despliegue conoce entorno, secretos y permisos; un DBA puede necesitar revisar operaciones sobre tablas grandes. Un buen proceso permite que cada responsabilidad intervenga sin exigir privilegios de esquema a la aplicación normal.

Profundización 2

Antes de aplicar cambios conviene comprobar conectividad, versión del esquema, espacio disponible, compatibilidad del artefacto y existencia de copia de seguridad cuando corresponda. Estas verificaciones reducen fallos evitables antes de entrar en la ventana de cambio.

Profundización 3

En despliegues con varias instancias puede existir un intervalo en el que código antiguo y nuevo comparten base. Los cambios expand/contract ayudan: primero se añade una estructura compatible, después se despliega código que la usa y, en una versión posterior, se retira lo antiguo.

Profundización 4

Una migración puede necesitar transformar datos existentes. Las operaciones grandes pueden bloquear tablas o tardar más de lo aceptable. En esos casos puede ser mejor separar migración de esquema y backfill de datos, usando procesos controlados e idempotentes.

Mini caso de aplicación

En un release con varias réplicas, el esquema puede desplegarse antes del código para que las instancias antiguas sigan funcionando. Después se publica código nuevo y, en otra versión, se eliminan columnas obsoletas.

Patrón de diagnóstico y preguntas de revisión

La técnica se revisa partiendo de una hipótesis concreta, una evidencia observable y una comprobación del estado final. Antes de convertir una recomendación en regla general conviene responder:

- ¿El destino está en una versión de esquema compatible con el artefacto?
- ¿La identidad de despliegue tiene solo los permisos necesarios?

- ¿Existe un backup o estrategia de recuperación acorde al riesgo?
- ¿Código antiguo y nuevo pueden convivir durante el despliegue?
- ¿Las migraciones de datos caben en la ventana prevista?
- ¿La verificación posterior confirma esquema y funcionamiento básico?

Decisiones y trade-offs

Opción	Qué aporta	Trade-off
--------	------------	-----------

Script SQL	Revisable por DBA	Muy auditable; hay que gestionar ejecución y secretos.
Bundle	Ejecutable autónomo	Cómodo para automatización; sigue necesitando control de permisos y conexión.
CLI	Simple en entornos controlados	Menos apropiado como mecanismo ad hoc en producción.
Runtime	La aplicación migra al arrancar	Riesgo de carreras y permisos en múltiples réplicas.

Errores frecuentes

- Ejecutar migraciones de producción desde el startup sin coordinación por defecto.
- Cambiar la tabla de historial heredada sin migrar su contenido.
- Guardar cadenas de producción en scripts o repositorio.
- Tratar downgrade como restauración de datos.

Preparar una migración para producción

Separar creación del artefacto y ejecución

En desarrollo es habitual ejecutar `dotnet ef database update` de forma interactiva. En producción interesa distinguir la construcción del artefacto de despliegue, su revisión y su ejecución. Un script SQL permite inspección y aprobación; un migration bundle empaqueta el ejecutor; la CLI puede ser válida en un job controlado. La elección depende de gobernanza, acceso al destino y requisitos de control operativo.

AceriaData no convierte `Database.Migrate()` durante el arranque normal de varias réplicas en la recomendación principal. Las migraciones cambian un recurso compartido y conviene que exista una única responsabilidad de despliegue, con credenciales y observabilidad apropiadas.

Historial de migraciones como parte del contrato

`EFMigrationsHistory` registra qué migraciones conoce la base como aplicadas. Cambiar su nombre mediante `MigrationsHistoryTable` después de años de uso sin mover los registros rompe la continuidad: EF Core puede ver una tabla vacía y considerar pendientes cambios ya existentes físicamente.

Por eso AceriaData conserva el nombre heredado. Personalizarlo es válido si se decide al principio o si se planifica una operación explícita que migre también la información histórica.

Preflight antes de tocar el esquema

Un despliegue sólido comprueba versión de aplicación, cadena de migraciones esperada, conectividad, permisos, espacio, estado de backups y posibles dependencias. También conviene conocer si hay cambios que puedan bloquear tablas durante periodos significativos. Un script generado no sustituye el análisis operacional del cambio concreto.

El preflight debe fallar antes de iniciar cambios irreversibles cuando una precondition crítica no se cumple. Cuanto antes se detecta una incompatibilidad, menos complejo es recuperar.

Verificación posterior

Después de aplicar la migración se revisan el historial y elementos de esquema relevantes. También se ejecutan smoke tests de las rutas afectadas. "El comando terminó con código 0" es una señal útil, pero no siempre basta para demostrar que la aplicación y el esquema esperado son compatibles.

Si un despliegue falla a mitad de la cadena, el diagnóstico parte de `EFMigrationsHistory` y del esquema real, no de lo que se esperaba que ocurriera. Esa evidencia permite decidir si continuar, aplicar una corrección o restaurar según el plan.

Downgrade y recuperación

Un script de downgrade ejecuta operaciones `Down`; puede eliminar columnas o datos. Por eso no debe equipararse automáticamente a un plan de recuperación. Para incidentes con riesgo de pérdida de información, el backup y la restauración verificada siguen siendo herramientas fundamentales. En ocasiones la estrategia más segura es una migración hacia delante que corrija el problema.

Lista de comprobación conceptual

- Elegir un artefacto de despliegue y versionarlo junto con la aplicación.
- Mantener credenciales de producción fuera del repositorio y del bundle.
- Ejecutar preflight antes del primer cambio.
- Verificar historial y esquema después de aplicar.
- Probar la recuperación, no solo escribirla en un documento.
- Tratar `Down` como código potencialmente destructivo.

Escenario de decisión: despliegue con revisión previa

Un equipo prepara una versión que incluye la migración final de concurrencia. En un entorno regulado, el DBA necesita revisar el SQL antes de ejecutar cambios. El script generado resulta apropiado porque puede almacenarse, revisarse y ejecutarse con permisos controlados. En un pipeline automatizado, un migration bundle puede simplificar la entrega del ejecutor sin exigir que la CLI de EF esté instalada en el destino.

Ambos caminos deben terminar en la misma pregunta: ¿la base quedó en la migración esperada y el esquema contiene los objetos previstos? El historial ofrece una parte de la respuesta y la verificación del esquema otra. Un proceso de despliegue maduro no se limita a que el comando no devuelva error.

Si algo falla, se reconstruye hasta qué migración avanzó la base y se consulta el estado físico. El plan de recuperación se elige con esa evidencia. Ejecutar automáticamente un downgrade puede ser más arriesgado que restaurar una copia o aplicar una corrección hacia delante si `Down` elimina información.

Anclaje en AceriaData

El concepto se implementa en `M05/PROJECT0/5.6`. La carpeta contiene su propia `AceriaData.sln` y continúa directamente desde el punto anterior. La práctica restaura, compila y ejecuta ese estado completo; cuando el punto no cambia el modelo, también se comprueba que no existan cambios de modelo pendientes.

Resumen de la teoría

- Distinguir estrategias de aplicación de migraciones en desarrollo y producción.
- Generar SQL revisable y migration bundles.
- Usar IMigrator en un laboratorio controlado.
- Conservar correctamente `__EFMigrationsHistory`.
- Entender el riesgo de cambiar MigrationsHistoryTable en una base ya desplegada.
- Preparar preflight, backup, verificación y reversión.

Punto 5.7 — Migraciones idempotentes y scripts SQL

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL, sin experiencia previa en ORMs ni en Entity Framework Core.

Proyecto: Este punto continúa el proyecto acumulativo `AceriaData` y estudia migraciones idempotentes y scripts sql sobre `.NET 8`, `Entity Framework Core 8` y `SQL Server LocalDB`.

Objetivos de aprendizaje

- Comprender qué significa idempotencia en un script de migraciones.
- Generar scripts completo, idempotente, de rango y downgrade desde la cadena real.
- Aplicar dos veces el mismo script idempotente sobre LocalDB y verificar el historial.
- Interpretar el papel de `__EFMigrationsHistory`.
- Reconocer que un script generado debe verificarse contra una base aislada.

- Tratar el downgrade como potencialmente destructivo.

Consideraciones técnicas en EF Core 8

La idempotencia se demuestra aplicando dos veces el mismo script a una base aislada y comparando historial y esquema.

Desarrollo teórico

Qué significa idempotencia

Un script idempotente puede ejecutarse sobre bases que se encuentran en distintos puntos de la misma cadena y aplica solo migraciones que todavía no figuran como aplicadas.

La palabra no se valida mirando un IF NOT EXISTS aislado: el laboratorio aplica el mismo artefacto dos veces y compara el historial y el esquema.

Script completo e idempotente

El script completo representa la secuencia desde el origen hasta el destino solicitado. El idempotente incorpora guardas basadas en `__EFMigrationsHistory` para omitir migraciones ya aplicadas.

Ambos son útiles, pero resuelven necesidades distintas. El completo es apropiado cuando se conoce el estado inicial; el idempotente tolera varios estados válidos de partida dentro de la cadena.

Scripts de rango

EF permite generar SQL entre dos migraciones concretas. AceriaData usa nombres que existen realmente: `M2_2_12_Architecture` y `M5_5_2_ConcurrencyTokens`.

Los ejemplos de migraciones usan los nombres que existen realmente en AceriaData para que los comandos del manual puedan repetirse sin reinterpretaciones.

Aplicación real con sqlcmd

El script PowerShell crea una LocalDB aislada y usa `sqlcmd` para aplicar el SQL. Después consulta `__EFMigrationsHistory` y `sys.columns` para comprobar migración final y esquema.

La opción `-I` se usa para `QUOTED_IDENTIFIER` conforme a las necesidades de los objetos generados por la cadena. La validación se detiene ante cualquier código de salida distinto de cero.

Segunda aplicación

Después de la primera ejecución se cuenta el historial. La segunda aplicación usa exactamente el mismo archivo y exige que el número de migraciones no cambie y que el esquema conserve la misma forma.

Ésta es una evidencia más fuerte que afirmar que el script parece idempotente.

Downgrade

La generación inversa permite estudiar SQL de Down. Puede ser útil en un rollback coordinado, pero debe revisarse por su potencial destructivo.

Un pipeline nunca debería asumir que poder generar un downgrade implica que es seguro ejecutarlo sobre datos valiosos.

Ejemplo ejecutable del concepto

POWERSHELL

```
dotnet ef migrations script --idempotent `
  --project $Infrastructure `
  --startup-project $Startup `
  --configuration Release `
  --output $Idempotent
& sqlcmd -S $Server -d $Database -E -I -b -i $Idempotent
$count1 = [int](Read-Scalar "SELECT COUNT(*) FROM dbo.__EFMigrationsHistory;")
& sqlcmd -S $Server -d $Database -E -I -b -i $Idempotent
$count2 = [int](Read-Scalar "SELECT COUNT(*) FROM dbo.__EFMigrationsHistory;")
```

El ejemplo refleja la misma línea de implementación que usa el estado validado del proyecto. La práctica trabaja con el archivo real y comprueba el resultado en SQL Server LocalDB.

Profundización y contexto de uso

Profundización 1

La principal ventaja de un script idempotente es poder llevar bases que están en diferentes puntos de la cadena hacia un mismo destino. El historial es la referencia que determina qué bloques deben ejecutarse. Esto no lo convierte en una herramienta de reconciliación de cualquier deriva manual.

Profundización 2

Un caso peligroso es que `__EFMigrationsHistory` diga que una migración está aplicada mientras un objeto fue modificado manualmente. El script puede omitir la operación porque confía en el historial. Por eso disciplina de despliegue y verificaciones de esquema siguen siendo necesarias.

Profundización 3

Un rango es útil cuando el release conoce exactamente desde qué versión parte el destino. Si hay múltiples estados posibles, el script idempotente suele ser más flexible. Ambos se generan desde la misma cadena.

Profundización 4

`$ErrorActionPreference = Stop` cubre errores de cmdlets, pero los ejecutables externos comunican su resultado mediante `$LASTEXITCODE`. El script convierte códigos distintos de cero en excepciones para que el pipeline no continúe tras un fallo.

Mini caso de aplicación

Una base A está dos migraciones por detrás y otra B solo una. El mismo script idempotente puede llevar ambas al destino porque consulta el historial antes de cada bloque. Si alguien alteró manualmente el esquema, esa idempotencia no repara automáticamente la deriva.

Patrón de diagnóstico y preguntas de revisión

La técnica se revisa partiendo de una hipótesis concreta, una evidencia observable y una comprobación del estado final. Antes de convertir una recomendación en regla general conviene responder:

- ¿Se aplicó realmente el mismo archivo dos veces?
- ¿El número de migraciones permanece estable tras la segunda ejecución?
- ¿La migración final es la esperada?
- ¿Se comprobaron objetos de esquema relevantes además del historial?
- ¿El script de rango usa nombres reales de la cadena?
- ¿El downgrade fue revisado por pérdida de datos?

Decisiones y trade-offs

Opción	Qué aporta	Trade-off
--------	------------	-----------

Completo	Estado inicial conocido	Más simple, menos tolerante a estados diferentes.
Idempotente	Varios estados válidos	Más flexible; debe probarse.
Rango	Cambio concreto	Útil para releases parciales.
Downgrade	Ejecuta Down	Puede perder datos.

Errores frecuentes

- Validar idempotencia solo leyendo el SQL.
- Usar nombres de migración que no existen en el repositorio.
- No ejecutar el mismo script dos veces.
- Aplicar un downgrade destructivo sin backup o revisión.

Validar de verdad la idempotencia

Idempotencia no es solo encontrar IF NOT EXISTS

Un script puede contener guardas condicionales y aun así no producir el esquema esperado. La prueba útil aplica el artefacto sobre una base conocida, verifica historial y esquema, y después ejecuta exactamente el mismo script una segunda vez. La segunda ejecución debe terminar correctamente y no duplicar las migraciones ya registradas.

AceriaData automatiza esa secuencia en LocalDB. El primer recuento de `__EFMigrationsHistory` se conserva y se compara con el segundo. También se verifican elementos de esquema asociados a la migración final de concurrencia.

Script completo, de rango e idempotente

El script completo describe la evolución desde el inicio hasta el destino seleccionado. El script de rango parte de una migración concreta y llega a otra. El idempotente incorpora condiciones basadas en el historial para poder ejecutarse contra bases que no estén necesariamente en la misma posición de la cadena.

Estos artefactos responden a necesidades diferentes. Un despliegue controlado puede conocer exactamente la versión origen y preferir un rango específico; una flota con distintas versiones puede necesitar un artefacto idempotente, siempre que las rutas intermedias hayan sido probadas.

Detectar drift

Si el historial dice que una migración está aplicada pero el objeto físico correspondiente no existe, hay drift entre historia y esquema. Un script idempotente puede decidir no volver a ejecutar una migración porque confía en el historial. Por eso validar solo el número de filas de `__EFMigrationsHistory` no es suficiente: hay que revisar también el esquema que importa al cambio.

El drift suele indicar modificaciones manuales, restauraciones incompletas o procedimientos de despliegue que no respetaron la cadena. La respuesta no debería ser borrar filas del historial de forma impulsiva, sino reconstruir qué ocurrió y aplicar una corrección controlada.

Artefacto revisado e inmutable

Una vez revisado y aprobado un script para una versión, cambiar silenciosamente su contenido dificulta saber qué SQL llegó a cada entorno. Una práctica operativa razonable es asociar el artefacto a la versión de aplicación y conservarlo inmutable tras la aprobación, regenerando una nueva versión cuando cambia la cadena.

Esto permite relacionar incidencias con el SQL concreto ejecutado y reduce diferencias entre staging y producción.

Downgrade de rango

Generar el rango inverso sirve para estudiar qué operaciones `Down` produciría EF Core. El archivo debe revisarse con especial atención a operaciones destructivas y transformaciones de datos. La disponibilidad de un comando que lo genera no lo convierte en una recuperación automática y segura.

Lista de comprobación conceptual

- Generar el script desde las migraciones reales del proyecto.
- Aplicarlo a una base aislada conocida.
- Verificar migración final y objetos de esquema.
- Ejecutarlo una segunda vez sin modificarlo.
- Confirmar que el historial no crece en la segunda ejecución.
- Investigar drift si historial y esquema no coinciden.

Escenario de decisión: varias bases en versiones distintas

Imaginemos tres instalaciones de AceriaData. Una ya tiene la migración de concurrencia, otra está en una migración anterior y una tercera parte de una base nueva. Un script idempotente puede consultar `__EFMigrationsHistory` y ejecutar únicamente los bloques que faltan en cada caso, siempre que las rutas intermedias representadas por la cadena se hayan probado.

La prueba del laboratorio aplica el mismo artefacto dos veces sobre una base aislada. El segundo pase debe dejar igual el número de migraciones registradas y el esquema comprobado. Esta prueba es más fuerte que buscar texto `IF NOT EXISTS` dentro del archivo: valida el comportamiento del artefacto contra SQL Server.

Si el historial no cambia pero el esquema no contiene una columna esperada, existe una inconsistencia distinta: la historia afirma algo que el esquema no refleja. Ese caso exige investigar drift y no simplemente volver a lanzar el mismo script esperando que la guarda idempotente deje de aplicarse.

Anclaje en AceriaData

El concepto se implementa en `M05/PROYECTO/5.7`. La carpeta contiene su propia `AceriaData.sln` y continúa directamente desde el punto anterior. La práctica restaura, compila y ejecuta ese estado completo; cuando el punto no cambia el modelo, también se comprueba que no existan cambios de modelo pendientes.

Resumen de la teoría

- Comprender qué significa idempotencia en un script de migraciones.
- Generar scripts completo, idempotente, de rango y downgrade desde la cadena real.
- Aplicar dos veces el mismo script idempotente sobre LocalDB y verificar el historial.
- Interpretar el papel de `__EFMigrationsHistory`.
- Reconocer que un script generado debe verificarse contra una base aislada.
- Tratar el downgrade como potencialmente destructivo.

Punto 5.8 — Migraciones en equipos: conflictos y buenas prácticas

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL, sin experiencia previa en ORMs ni en Entity Framework Core.

Proyecto: Este punto continúa el proyecto acumulativo `AceriaData` y estudia migraciones en equipos: conflictos y buenas prácticas sobre .NET 8, Entity Framework Core 8 y SQL Server LocalDB.

Objetivos de aprendizaje

- Comprender cómo aparecen árboles de migraciones divergentes en equipos.
- Entender por qué renombrar archivos no fusiona los metadatos de una migración.
- Regenerar una migración propia sobre el snapshot ya fusionado.
- Usar `has-pending-model-changes` como comprobación automática.
- Aplicar la cadena resultante sobre una base aislada y verificar esquema e historial.
- Distinguir migración local no compartida de migración ya compartida o aplicada.

Consideraciones técnicas en EF Core 8

Una migración paralela no se arregla renombrándola. El cambio propio se regenera sobre el snapshot fusionado y se valida sobre SQL Server.

Desarrollo teórico

Cómo nace un conflicto de migraciones

Dos desarrolladores pueden partir del mismo snapshot y cada uno modificar el modelo. Si ambos generan una migración antes de integrar al otro, las dos migraciones representan futuros distintos del mismo estado inicial.

El problema no es solo el timestamp o el nombre: el archivo `Designer` y el snapshot codifican el modelo conocido en el momento de generación.

Por qué renombrar no fusiona

Cambiar el nombre o el orden aparente de archivos no recalcula los metadatos de una migración. Una migración B generada en paralelo puede seguir sin conocer la propiedad introducida por A.

El laboratorio lee el `Designer` de B y exige que no contenga `EquipoRevisionA`. Esa evidencia demuestra por qué un simple renombrado sería una falsa resolución.

Regenerar sobre el modelo fusionado

La estrategia correcta para una migración propia no compartida es conservar el cambio de modelo, incorporar el trabajo del compañero y regenerar la migración sobre el snapshot ya actualizado.

El `Designer` regenerado debe representar A+B. La práctica lo comprueba antes de tocar la base de datos.

has-pending-model-changes

EF Core 8 incorpora un comando que devuelve error si el modelo actual no coincide con el snapshot de la última migración. Es adecuado como comprobación automática para detectar cambios de modelo olvidados.

El comando no sustituye la prueba de aplicar la cadena. Por eso el laboratorio también crea una LocalDB aislada y ejecuta database update.

Verificar esquema e historial

Después de aplicar la cadena fusionada se comprueba con SQL Server que existen EquipoRevisionA y EquipoRevisionB. También se valida que __EFMigrationsHistory contiene TeamA y TeamBRegenerated.

La verificación conjunta evita aceptar una solución que solo compila pero no produce el esquema esperado.

Migraciones ya compartidas

Una migración que ya llegó a una base compartida tiene una historia distinta de una migración local. Borrarla o reescribirla unilateralmente rompe la correspondencia entre repositorio y bases desplegadas.

En ese caso se coordina rollback cuando es seguro o se crea una migración correctiva que avance desde el estado ya publicado.

Ejemplo ejecutable del concepto

El script prepara dos copias desechables del mismo estado inicial. Sobre esas copias ejecuta estas operaciones reales:

```
Add-TeamProperty $BranchA "EquipoRevisionA"
Add-Migration $BranchA "M5_5_8_TeamA"
Add-TeamProperty $BranchB "EquipoRevisionB"
Add-Migration $BranchB "M5_5_8_TeamBParallel"
Copy-Item $BranchA -Destination $Merged -Recurse -Force
Add-TeamProperty $Merged "EquipoRevisionB"
Add-Migration $Merged "M5_5_8_TeamBRegenerated"
```

POWERSHELL

El ejemplo refleja la misma línea de implementación que usa el estado validado del proyecto. La práctica trabaja con el archivo real y comprueba el resultado en SQL Server LocalDB.

Profundización y contexto de uso

Profundización 1

Git puede fusionar dos archivos sin marcar conflicto y, aun así, la secuencia de migraciones ser semánticamente incorrecta. También puede marcar un conflicto textual en el snapshot que sea sencillo de resolver una vez claro el modelo combinado.

Profundización 2

Cada migración incluye metadatos generados a partir del modelo objetivo. Esos metadatos no se actualizan solo porque el nombre del archivo cambie. El laboratorio inspecciona el Designer de B paralela para hacer visible este hecho.

Profundización 3

Cuando la última migración es propia, no está aplicada en bases compartidas y puede retirarse con seguridad, migrations remove permite volver al snapshot anterior sin perder necesariamente el cambio que sigue en el código de modelo.

Profundización 4

El orden seguro no consiste en hacer que timestamps queden ordenados, sino en conseguir que cada nueva migración se genere desde el snapshot que contiene todas las migraciones anteriores ya integradas.

Mini caso de aplicación

A añade una columna y genera su migración. B añade otra desde el mismo ancestro. Después de integrar A, B conserva su cambio de modelo pero regenera su migración. El nuevo Designer representa A+B y la cadena se valida sobre una base limpia.

Patrón de diagnóstico y preguntas de revisión

La técnica se revisa partiendo de una hipótesis concreta, una evidencia observable y una comprobación del estado final. Antes de convertir una recomendación en regla general conviene responder:

- ¿La migración conflictiva ya fue compartida o aplicada en una base común?
- ¿Se conserva el cambio de modelo antes de retirar una migración propia no publicada?
- ¿La migración regenerada conoce los cambios incorporados del compañero?
- ¿El snapshot final representa el modelo combinado?
- ¿has-pending-model-changes termina sin diferencias?
- ¿La cadena completa se ejecuta sobre SQL Server y produce ambos cambios?

Decisiones y trade-offs

Opción	Qué aporta	Trade-off
--------	------------	-----------

Renombrar	Solo cambia apariencia u orden	No fusiona metadata ni snapshot.
Regenerar	Recrea migración sobre modelo integrado	Correcto para migración propia no compartida.
Correctiva	Avanza desde estado ya publicado	Adecuada cuando una migración ya salió a entornos compartidos.

Errores frecuentes

- Renombrar una migración paralela y dar el conflicto por resuelto.
- Editar una migración ya aplicada en una base compartida.
- No ejecutar has-pending-model-changes.
- Validar solo que el proyecto compila, sin aplicar la cadena.

Resolver divergencias de migraciones en equipo

El problema está en la historia del modelo

Cuando dos desarrolladores generan migraciones desde el mismo snapshot, cada una describe un cambio desde ese estado común. La migración de B no puede contener conocimiento del cambio de A que todavía no existía en su rama. El nombre del archivo y su prefijo temporal ordenan artefactos, pero no reescriben el modelo que quedó capturado en el `.Designer.cs`.

Por eso "renombrar para que quede después" no fusiona semánticamente las ramas. Puede dar apariencia de orden y mantener metadatos inconsistentes.

Reconstruir el ancestro común

El diagnóstico empieza identificando la última migración y snapshot que ambos desarrolladores compartían. Después se comparan cambios de modelo de A y B. Saber qué parte pertenece a cada rama permite decidir si una migración local puede retirarse y regenerarse o si ya forma parte de una historia compartida que debe conservarse.

AceriaData reproduce este proceso con copias desechables: Rama A y Rama B paralela nacen del mismo estado. El `.Designer.cs` de B se comprueba para demostrar que desconoce A.

Regenerar cuando todavía es seguro

Si la migración propia de B no se ha compartido ni aplicado en una base compartida, B puede conservar su cambio de código, incorporar A y generar una nueva migración sobre el snapshot fusionado. La nueva metadata conoce ambos cambios y se convierte en la sucesora coherente de A.

Después se ejecuta `has-pending-model-changes`. Esa comprobación detecta si el modelo actual sigue conteniendo un cambio que no está representado por la cadena de migraciones. A continuación se aplica todo a una base limpia y se comprueban las columnas y el historial.

Cuando la migración ya salió de la rama local

Una migración aplicada por otros desarrolladores, por integración continua o por un entorno compartido deja de ser un archivo privado que se puede reescribir unilateralmente. Modificarla crea distintas interpretaciones de un mismo identificador de migración. En ese punto se coordina un rollback cuando es seguro o, con más frecuencia, se añade una migración correctiva que lleve todos los entornos hacia un estado consistente.

La regla práctica no es "nunca borrar una migración", sino considerar su ámbito de publicación. Antes de compartirla puede regenerarse; después de formar parte de la historia común requiere coordinación.

Revisar una migración como código

La revisión incluye `Up`, `Down`, metadata, snapshot y efecto previsto sobre datos. También debe comprobar nombres de columnas, nulabilidad, defaults y operaciones potencialmente costosas. El merge de Git puede resolver conflictos de texto y aun dejar una cadena conceptualmente incorrecta; por eso las pruebas sobre una base limpia siguen siendo necesarias.

Lista de comprobación conceptual

- Identificar el snapshot común de las ramas.
- No usar un renombrado como sustituto de regeneración.
- Regenerar solo migraciones todavía locales/no compartidas.
- Ejecutar `has-pending-model-changes` tras la fusión.
- Aplicar la cadena completa sobre una base aislada.
- Verificar tanto el esquema final como `EFMigrationsHistory`.

Escenario de decisión: dos ramas crean migraciones en paralelo

A y B parten de la misma versión. A añade `EquipoRevisionA`; B, sin conocer ese cambio, añade `EquipoRevisionB`. Cada desarrollador genera una migración válida respecto de su propio snapshot. Al unir las ramas, el problema no es solo que los nombres puedan ordenarse de una manera u otra: la metadata de la migración B fue producida sin el cambio de A.

Si B todavía no ha compartido ni aplicado su migración, puede conservar el cambio de modelo, incorporar A y regenerar su migración. La nueva metadata representa A+B. El laboratorio verifica precisamente esa diferencia inspeccionando los designers y aplicando la cadena fusionada en LocalDB.

Si B ya publicó su migración, reescribirla unilateralmente crea dos historias con el mismo identificador conceptual. En ese escenario se coordina una corrección. La frontera relevante es la publicación del historial, no una regla mecánica basada únicamente en si el archivo está o no en la carpeta local.

Anclaje en AceriaData

El concepto se implementa en `M05/PROYECTO0/5.8`. La carpeta contiene su propia `AceriaData.sln` y continúa directamente desde el punto anterior. La práctica restaura, compila y ejecuta ese estado completo; cuando el punto no cambia el modelo, también se comprueba que no existan cambios de modelo pendientes.

Resumen de la teoría

- Comprender cómo aparecen árboles de migraciones divergentes en equipos.
- Entender por qué renombrar archivos no fusiona los metadatos de una migración.
- Regenerar una migración propia sobre el snapshot ya fusionado.
- Usar `has-pending-model-changes` como comprobación automática.
- Aplicar la cadena resultante sobre una base aislada y verificar esquema e historial.
- Distinguir migración local no compartida de migración ya compartida o aplicada.

Punto 5.9 — Patrón Repositorio y Unidad de Trabajo en aplicaciones empresariales

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL, sin experiencia previa en ORMs ni en Entity Framework Core.

Proyecto: Este punto continúa el proyecto acumulativo AceriaData y estudia patrón repositorio y unidad de trabajo en aplicaciones empresariales sobre .NET 8, Entity Framework Core 8 y SQL Server LocalDB.

Objetivos de aprendizaje

- Comprender el papel de Repository y Unit of Work como decisión arquitectónica, no requisito universal.
- Mantener Application desacoplada de EF Core mediante interfaces específicas.

- Coordinar varias operaciones con una unidad de trabajo.
- Usar IDbContextFactory para crear contextos independientes bajo demanda.
- Introducir xUnit y Moq sin mockear DbSet ni el proveedor LINQ.
- Evaluar el patrón por responsabilidades y equivalencia funcional, no por cifras de coste inventadas.

Consideraciones técnicas en EF Core 8

Repository/UoW es una decisión de arquitectura de AceriaData. EF Core ya proporciona capacidades equivalentes en DbContext/DbSet.

Desarrollo teórico

DbContext ya contiene ideas de Repository y UoW

DbSet expone operaciones sobre un conjunto de entidades y DbContext coordina cambios y SaveChanges. Por eso añadir interfaces Repository y Unit of Work no es un requisito de EF Core.

AceriaData las conserva porque Application no debe depender de Infrastructure/EF Core y porque expresa operaciones específicas mediante contratos de aplicación.

Repository genérico mínimo

El Repository<T> de AceriaData mantiene operaciones comunes deliberadamente pequeñas: buscar, listar, agregar y eliminar. Las consultas de negocio viven en repositorios específicos.

Un repositorio genérico que intenta reproducir toda la API de DbSet suele añadir complejidad sin ocultar realmente el proveedor.

IQueryable y frontera arquitectónica

Exponer IQueryable puede ser útil en algunos diseños, pero desplaza la composición y conocimiento del proveedor hacia el consumidor. AceriaData decide no hacerlo en su frontera de Application.

Esto es una decisión de diseño, no una ley universal. Un proyecto que expone IQueryable dentro de una misma capa puede tener un trade-off distinto.

Unidad de trabajo

IUnidadDeTrabajo coordina Ordenes y Detalles sobre un mismo AceriaDbContext y ofrece una única operación Guardar. Así el caso de uso puede expresar una operación coherente sin conocer EF Core.

No se debe duplicar transacciones innecesarias: SaveChanges ya implementa una unidad atómica para los cambios pendientes del contexto.

IDbContextFactory

IDbContextFactory permite crear contextos independientes bajo demanda, útil cuando el ciclo de vida del consumidor no coincide con un scope de petición o cuando se necesitan unidades separadas.

Cada contexto debe disponer correctamente sus recursos; la fábrica no convierte DbContext en un singleton seguro para hilos.

Testing con Moq

Los unit tests de 5.9 sustituyen IOrdenRepository, no DbSet. De ese modo prueban la lógica de Application sin intentar simular el proveedor LINQ de EF Core.

Los tests con base real llegan en 5.11. Mantener estas capas de prueba separadas hace más claro qué comportamiento valida cada una.

Ejemplo ejecutable del concepto

```
public class Repositorio<T> : IRepository<T> where T : class
{
    protected readonly AceriaDbContext _context;
    protected readonly DbSet<T> _dbSet;
    public Repositorio(AceriaDbContext context)
    {
        _context = context;
        _dbSet = context.Set<T>();
    }
}
```

CSHARP

```

}
public virtual T? ObtenerPorId(int id) => _dbSet.Find(id);
public virtual List<T> ObtenerTodas() => _dbSet.ToList();
public virtual void Agregar(T entidad) => _dbSet.Add(entidad);
public virtual void Eliminar(T entidad) => _dbSet.Remove(entidad);
}

```

El ejemplo refleja la misma línea de implementación que usa el estado validado del proyecto. La práctica trabaja con el archivo real y comprueba el resultado en SQL Server LocalDB.

Profundización y contexto de uso

Profundización 1

Una interfaz demasiado genérica puede esconder capacidades de EF Core sin expresar mejor el dominio. Una interfaz demasiado específica puede multiplicar métodos casi idénticos. El diseño útil está donde Application expresa operaciones que entiende e Infrastructure conserva libertad para implementarlas eficientemente.

Profundización 2

Cuando una operación solo necesita lectura, un repositorio puede proyectar directamente al DTO de aplicación si las dependencias están orientadas correctamente. Esto evita materializar entidades completas y exponer IQueryable más allá de la frontera elegida.

Profundización 3

Para escritura conviene que la unidad de trabajo preserve reglas del agregado y confirme juntos los cambios relacionados. Si cada repositorio llama a SaveChanges internamente, la aplicación pierde control sobre la atomicidad de una operación que usa varios repositorios.

Profundización 4

El repositorio no debe ocultar un DbContext singleton. Sus instancias participan en el mismo scope cuando deben formar una unidad. Para procesos que necesitan contextos separados, IDbContextFactory hace explícita la creación bajo demanda.

Mini caso de aplicación

Un caso de uso necesita órdenes pendientes del cliente con su resumen. DbContext puede expresarlo directamente; un repositorio específico puede ofrecer una operación con nombre de dominio y proyectar solo lo necesario. La elección depende de la arquitectura.

Patrón de diagnóstico y preguntas de revisión

La técnica se revisa partiendo de una hipótesis concreta, una evidencia observable y una comprobación del estado final. Antes de convertir una recomendación en regla general conviene responder:

- ¿La interfaz expresa operaciones que entiende Application?
- ¿Los repositorios comparten el mismo contexto cuando deben confirmar juntos?
- ¿Quién decide cuándo llamar a SaveChanges?
- ¿Se está ocultando EF Core por una necesidad arquitectónica o solo por costumbre?
- ¿Los tests unitarios prueban reglas, no traducción SQL?
- ¿Las consultas específicas conservan capacidad de proyectar eficientemente?

Decisiones y trade-offs

Opción	Qué aporta	Trade-off
--------	------------	-----------

DbContext directo	Menos abstracción	Válido si la arquitectura permite depender de EF Core.
Repository específico	Frontera de aplicación	Añade contratos y mantenimiento.
Repository genérico excesivo	Replica DbSet	Puede añadir delegación sin semántica.

Errores frecuentes

- Convertir Repository en requisito universal.
- Exponer una API genérica que replica DbSet completo.

- Mockear DbSet para simular SQL.
- Atribuir un coste fijo al patrón sin benchmark controlado.

Evaluar Repository y Unit of Work sin dogmas

Qué problema debe resolver la abstracción

DbContext ya coordina cambios y DbSet<T> ya ofrece acceso a conjuntos de entidades. Añadir interfaces propias solo tiene sentido si crean una frontera útil: lenguaje de aplicación, aislamiento de dependencias, operaciones específicas del dominio o capacidad de sustituir colaboradores en pruebas unitarias. Un repositorio que únicamente renombra Add, Find y ToList puede añadir capas sin reducir complejidad.

En AceriaData la razón principal es arquitectónica: Application no referencia EF Core. Sus casos de uso dependen de interfaces propias y Infrastructure aporta las implementaciones.

Evitar filtrar IQueryable fuera de la frontera

Exponer IQueryable<T> desde una interfaz que pretende ocultar EF Core puede trasladar detalles de consultas, includes y proveedor a capas superiores. El consumidor termina componiendo una consulta cuyo comportamiento solo puede entenderse con el ORM que la ejecutará. En ese caso la abstracción es nominal, no efectiva.

Una alternativa es definir consultas con intención, DTOs o especificaciones bien delimitadas. Tampoco hay que convertir cada consulta en un método sin criterio: el diseño busca una frontera comprensible, no multiplicar delegaciones triviales.

Unidad de trabajo y un único SaveChanges

Una unidad de trabajo propia puede agrupar varios repositorios que comparten el mismo DbContext y ofrecer una operación de confirmación. El valor aparece cuando el caso de uso necesita coordinar cambios en varias entidades dentro de la misma unidad. Si cada repositorio crea un contexto independiente y guarda de inmediato, se pierde esa coordinación.

Por eso el ciclo de vida del contexto forma parte del patrón. El contenedor DI debe hacer que repositorios y unidad de trabajo del mismo scope usen la misma instancia cuando esa es la semántica deseada.

IDbContextFactory

Procesos de larga duración, workers o componentes que no encajan en un scope HTTP pueden necesitar crear contextos cortos bajo demanda. IDbContextFactory<AceriaDbContext> proporciona ese mecanismo sin convertir un único DbContext en singleton. Cada contexto creado mantiene su propio tracker y debe ser dispuesto cuando termina la unidad de trabajo.

No debe confundirse la factory con una obligación de crear un contexto por método. La unidad de trabajo define el límite útil.

Unit tests con Moq

Las pruebas de Application simulan IOrdenRepositorio y verifican reglas como filtrar órdenes pendientes o delegar un registro. No mockean DbSet ni intentan reproducir el traductor LINQ. Esa separación mantiene el unit test pequeño: la semántica específica de EF se valida en pruebas de integración posteriores.

Lista de comprobación conceptual

- Explicar qué dependencia o responsabilidad justifica el repositorio propio.
- Evitar que la interfaz filtre APIs específicas de EF Core si el objetivo es desacoplar.
- Confirmar que repositorios coordinados comparten la misma unidad de trabajo.
- Usar contextos cortos con IDbContextFactory en procesos largos.
- Probar lógica de Application con mocks de sus abstracciones, no del ORM.
- Medir costes solo con un benchmark diseñado para ello, no con cifras prefijadas.

Escenario de decisión: una frontera que aporta valor

Un caso de uso necesita registrar una orden y su detalle como una sola operación. Con interfaces de Application, la unidad de trabajo puede exponer los repositorios necesarios y una confirmación única, mientras Infrastructure mantiene el DbContext concreto. El caso de uso conoce el lenguaje de aplicación y no necesita importar Microsoft.EntityFrameworkCore.

Ahora imaginemos un repositorio genérico que devuelve `IQueryable<T>` y obliga al caso de uso a llamar `Include`, `AsNoTracking` y métodos específicos del proveedor. La capa existe, pero la dependencia conceptual de EF ha atravesado la frontera. Esa situación sirve para revisar si la abstracción realmente simplifica responsabilidades o solo añade delegaciones.

Los unit tests con Moq muestran el beneficio cuando la lógica puede ejercerse a través de una interfaz pequeña. Las consultas y la persistencia real siguen necesitando tests de integración; la abstracción no convierte el comportamiento del ORM en algo que pueda validarse completamente con mocks.

Anclaje en AceriaData

El concepto se implementa en `M05/PROYECTO0/5.9`. La carpeta contiene su propia `AceriaData.sln` y continúa directamente desde el punto anterior. La práctica restaura, compila y ejecuta ese estado completo; cuando el punto no cambia el modelo, también se comprueba que no existan cambios de modelo pendientes.

Resumen de la teoría

- Comprender el papel de Repository y Unit of Work como decisión arquitectónica, no requisito universal.
- Mantener Application desacoplada de EF Core mediante interfaces específicas.
- Coordinar varias operaciones con una unidad de trabajo.
- Usar IDbContextFactory para crear contextos independientes bajo demanda.
- Introducir xUnit y Moq sin mockear DbSet ni el proveedor LINQ.
- Evaluar el patrón por responsabilidades y equivalencia funcional, no por cifras de coste inventadas.

Punto 5.10 — Logging y diagnóstico en Entity Framework Core

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL, sin experiencia previa en ORMs ni en Entity Framework Core.

Proyecto: Este punto continúa el proyecto acumulativo AceriaData y estudia logging y diagnóstico en entity framework core sobre .NET 8, Entity Framework Core 8 y SQL Server LocalDB.

Objetivos de aprendizaje

- Configurar ILogger e ILoggerFactory con categorías de EF Core.
- Usar Serilog estructurado y scopes.
- Configurar rotación de archivos por tamaño y retención.
- Implementar un observador completo de DiagnosticListener.
- Capturar EventCounters de EF Core 8.
- Integrar Application Insights en una aplicación Console/Worker.
- Entender el papel de dotnet-counters y la diferencia entre diagnóstico local y backend de observabilidad.

Consideraciones técnicas en EF Core 8

El punto usa ILogger, Serilog, DiagnosticListener, EventCounters de EF Core 8 y Application Insights WorkerService, con rotación real de archivos.

Desarrollo teórico

Logging estructurado

ILogger permite registrar eventos con plantillas y propiedades en lugar de concatenar cadenas. Serilog conserva esas propiedades y puede enviarlas a múltiples sinks.

AceriaData añade un scope con Modulo y Punto y registra NumeroOrden, Cliente y Filas como datos estructurados. La observabilidad mejora cuando los campos pueden filtrarse y agregarse.

Categorías de EF Core

Microsoft.EntityFrameworkCore.Database.Command, Update y Query permiten controlar el nivel por área. Registrar todo en Debug en producción puede generar volumen y exposición innecesarios.

El laboratorio fija filtros explícitos y mantiene EnableSensitiveDataLogging desactivado por defecto.

Rotación de archivo

El sink de archivo combina rollingInterval diario, fileSizeLimitBytes, rollOnFileSizeLimit y retainedFileCountLimit. El punto fuerza suficiente salida para comprobar que se crean varios archivos y que la retención máxima se respeta.

La validación automática evita documentar una configuración que nunca se ha probado físicamente.

DiagnosticListener

DiagnosticListener expone eventos de diagnóstico dentro del proceso. El observador se suscribe primero a AllListeners, selecciona Microsoft.EntityFrameworkCore y filtra eventos relacionados con Command y SaveChanges.

Las subscriptions son IDisposable y se liberan. Ignorar el ciclo de vida puede dejar observadores activos más tiempo del previsto.

EventCounters de EF Core 8

EventListener detecta el EventSource Microsoft.EntityFrameworkCore y solicita EventCounters cada segundo. Los payloads pueden contener Mean o Increment según el contador.

Para EF Core 8 el curso usa EventCounters y dotnet-counters. No atribuye a EF8 APIs de métricas introducidas en versiones posteriores.

Application Insights y Azure Monitor

En una aplicación de consola/worker se usa AddApplicationInsightsTelemetryWorkerService. El laboratorio instala un canal local que recibe telemetría para poder validar el SDK sin una suscripción Azure ni tráfico externo.

En arquitecturas actuales Azure Monitor también puede recibir telemetría vía OpenTelemetry. Esa ruta se explica como contexto, sin convertir este punto en un curso de Azure.

Ejemplo ejecutable del concepto

```
public void OnNext(DiagnosticListener listener)
{
    if (listener.Name == "Microsoft.EntityFrameworkCore")
        _subscriptions.Add(listener.Subscribe(this, IsEnabled));
}
private static bool IsEnabled(string eventName, object? arg1, object? arg2) =>
    eventName.Contains("Command", StringComparison.Ordinal) ||
    eventName.Contains("SaveChanges", StringComparison.Ordinal);
```

CSHARP

El ejemplo refleja la misma línea de implementación que usa el estado validado del proyecto. La práctica trabaja con el archivo real y comprueba el resultado en SQL Server LocalDB.

Profundización y contexto de uso

Profundización 1

En una aplicación real interesa asociar comandos, casos de uso y peticiones. Los scopes de ILogger permiten añadir propiedades comunes a todos los mensajes emitidos dentro de una operación. Un mensaje de texto sin contexto puede no bastar para relacionar una consulta con el caso de uso que la originó.

Profundización 2

Escribir a archivo sin límites puede llenar el disco. Serilog permite combinar RollingInterval, fileSizeLimitBytes, rollOnFileSizeLimit y retainedFileCountLimit. La configuración exacta de producción depende del volumen y normalmente se integra con un sistema centralizado.

Profundización 3

No todos los eventos necesitan observarse. El predicado IsEnabled filtra Command y SaveChanges, reduciendo el trabajo del observador. Una instrumentación demasiado detallada puede añadir volumen; observabilidad también necesita presupuestos de coste.

Profundización 4

Los counters ofrecen una vista agregada periódica, no una traza de cada operación. Sirven para observar tendencias del proceso, mientras logs y eventos detallados ayudan a investigar una operación concreta.

Mini caso de aplicación

Una incidencia informa de lentitud al guardar órdenes. EventCounters muestran tendencia general; logs estructurados identifican la operación; DiagnosticSource observa eventos de comandos y SaveChanges; telemetría correlacionada puede unir señales. Ninguna fuente aislada responde todo.

Patrón de diagnóstico y preguntas de revisión

La técnica se revisa partiendo de una hipótesis concreta, una evidencia observable y una comprobación del estado final. Antes de convertir una recomendación en regla general conviene responder:

- ¿Las categorías de EF tienen niveles adecuados para el entorno?
- ¿La rotación evita crecimiento ilimitado de archivos?
- ¿Se liberan las subscriptions de DiagnosticListener?
- ¿Los EventCounters observados pertenecen realmente a EF Core 8?
- ¿Los logs contienen valores sensibles?
- ¿La telemetría se valida en el punto donde se hace la afirmación?

Decisiones y trade-offs

Opción	Qué aporta	Trade-off
--------	------------	-----------

ILogger	Abstracción estándar	Buen punto de entrada para la aplicación.
DiagnosticListener	Eventos internos detallados	Requiere filtrar y gestionar subscriptions.
EventCounters	Métricas runtime EF8	Apropriados para observación agregada.
Application Insights	Backend/SDK	Necesita configuración y estrategia de privacidad.

Errores frecuentes

- Habilitar datos sensibles en logs de producción por defecto.
- Suscribirse a DiagnosticListener sin Dispose.
- Usar AddApplicationInsightsTelemetry() de ASP.NET Core en una consola como si fuera equivalente.
- Confundir EventCounters de EF8 con métricas de versiones posteriores.

Construir una cadena de observabilidad útil

Logs estructurados frente a cadenas de texto

ILogger permite registrar plantillas y propiedades separadas. Con "Orden {NumeroOrden} ..." el backend puede conservar NumeroOrden como dimensión consultable, en lugar de recibir solo una cadena final. Los scopes añaden contexto común —por ejemplo módulo, operación o correlación— a varios mensajes relacionados.

La estructura facilita buscar todas las operaciones de una orden o comparar errores por tipo sin depender de expresiones regulares sobre texto libre. También exige gobernanza: propiedades con datos sensibles no deben registrarse indiscriminadamente.

Filtros y volumen

EF Core publica categorías con granularidad distinta. Activar todo a nivel muy detallado en producción puede generar demasiado volumen, coste y ruido. La configuración debe seleccionar categorías y niveles útiles para el diagnóstico esperado. Database.Command ayuda a observar comandos; otras categorías exponen materialización, cambios y advertencias.

EnableSensitiveDataLogging() merece una decisión independiente. Puede revelar valores de parámetros y por eso AceriaData no lo deja activado por defecto.

DiagnosticListener

Un observador completo se suscribe primero a `DiagnosticListener.AllListeners`, identifica el listener de EF Core y después filtra eventos de interés. También conserva las subscriptions para liberarlas con `Dispose`. Suscribirse a un objeto equivocado o no disponer la suscripción puede producir pérdida de eventos o recursos retenidos.

`DiagnosticSource` ofrece detalles de eventos; no reemplaza automáticamente a logs o métricas. Es otra fuente que puede alimentar diagnóstico o instrumentación especializada.

EventCounters y dotnet-counters

EF Core 8 publica contadores mediante `EventSource`. Un `EventListener` puede habilitarlos con un intervalo, leer `Mean` o `Increment` y almacenar valores observados. Desde fuera del proceso, `dotnet-counters` permite monitorizarlos sin insertar lógica de presentación dentro de la aplicación.

Un contador agregado ayuda a observar tendencia, pero no identifica por sí solo qué solicitud concreta produjo un pico. Por eso se combina con logs y correlación.

Application Insights en Console/Worker

La integración adecuada para una aplicación de consola/worker usa `AddApplicationInsightsTelemetryWorkerService`. El laboratorio emplea un canal local para comprobar que el SDK produce telemetría sin depender de una suscripción Azure ni enviar datos reales. Ver una llamada a `TrackEvent` no demostraría por sí sola que un backend remoto recibió el evento.

En arquitecturas actuales también puede usarse `OpenTelemetry` y exportar a Azure Monitor u otros backends. El concepto docente es separar instrumentación, transporte y almacenamiento/consulta.

Lista de comprobación conceptual

- Registrar propiedades estructuradas y una correlación útil.
- Filtrar categorías y niveles según el objetivo.
- Mantener datos sensibles fuera de logs normales.
- Disponer subscriptions de `DiagnosticListener`.
- Interpretar contadores como agregados, no como trazas individuales.
- Verificar el canal/exportador antes de afirmar recepción remota de telemetría.

Escenario de decisión: investigar una escritura lenta

Un usuario informa de que guardar una orden tarda más de lo esperado. El log estructurado puede indicar qué operación y qué orden estaban involucradas; las categorías de EF pueden mostrar comandos; `DiagnosticSource` puede contar eventos de `SaveChanges` y comandos; `EventCounters` aportan una visión agregada del proceso. Cada señal responde una parte diferente del diagnóstico.

Si los logs incluyen una propiedad de correlación común, es posible seguir la secuencia sin depender de texto libre. Si el volumen de `Database.Command` es excesivo, se ajustan filtros o niveles. Si hace falta analizar valores de parámetros en desarrollo, `EnableSensitiveDataLogging` se activa de forma controlada y no se deja como configuración normal de producción.

El laboratorio de Application Insights valida el SDK mediante un canal local. Esta distinción evita afirmar que un servicio remoto recibió datos solo porque `TrackEvent` fue invocado. En producción también se comprobarían configuración del exportador, conectividad y recepción en el backend de observabilidad elegido.

Anclaje en AceriaData

El concepto se implementa en `M05/PROJECT0/5.10`. La carpeta contiene su propia `AceriaData.sln` y continúa directamente desde el punto anterior. La práctica restaura, compila y ejecuta ese estado completo; cuando el punto no cambia el modelo, también se comprueba que no existan cambios de modelo pendientes.

Resumen de la teoría

- Configurar `ILogger` e `ILoggerFactory` con categorías de EF Core.
- Usar Serilog estructurado y scopes.
- Configurar rotación de archivos por tamaño y retención.
- Implementar un observador completo de `DiagnosticListener`.
- Capturar `EventCounters` de EF Core 8.
- Integrar Application Insights en una aplicación Console/Worker.

Punto 5.11 — Testing con EF Core

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL, sin experiencia previa en ORMs ni en Entity Framework Core.

Proyecto: Este punto continúa el proyecto acumulativo **AceriaData** y estudia testing con ef core sobre .NET 8, Entity Framework Core 8 y SQL Server LocalDB.

Objetivos de aprendizaje

- Separar unit tests, provider-behavior tests e integration tests.
- Usar xUnit y Moq para probar Application sin EF Core.
- Entender las limitaciones de InMemory y SQLite como sustitutos de SQL Server.
- Ejecutar migraciones reales sobre SQL Server LocalDB.
- Aislar datos con Respawn sin borrar __EFMigrationsHistory.
- Validar rowversion real de SQL Server.
- Usar WebApplicationFactory contra una API real y una base de pruebas real.

Consideraciones técnicas en EF Core 8

La suite separa unit, provider behavior, integración SQL Server y HTTP. Las pruebas SQL Server usan migraciones reales y Respawn conserva el historial.

Desarrollo teórico

Pirámide de pruebas por intención

Una prueba útil especifica qué capa y qué comportamiento pretende validar. Unit tests de Application deben ser rápidos y no necesitar EF. Provider-behavior tests muestran diferencias de proveedores. Integration tests validan SQL Server y migraciones reales.

Mezclar todas estas metas en un único tipo de test produce falsos niveles de confianza.

Moq sin DbSet

RepositoryPatternTests configura IOrdenRepositorio con Moq y verifica llamadas. La consulta LINQ de EF Core no se simula; se sustituye la frontera que Application ya define.

Este enfoque evita recrear un proveedor LINQ falso que diverge del comportamiento de SQL Server.

InMemory y SQLite

InMemory no es relacional y puede aceptar datos que una base relacional rechazaría. SQLite sí implementa restricciones relacionales, pero tiene SQL, tipos y collation distintos de SQL Server.

El test de proveedor usa un modelo mínimo para demostrar una FK: InMemory acepta el hijo huérfano; SQLite, con foreign_keys habilitado, rechaza la escritura.

SQL Server LocalDB y migraciones

SqlServerDatabaseFixture crea una base única, construye AceriaDbContext con el ensamblado de migraciones y ejecuta MigrateAsync. No usa EnsureCreated.

Así la prueba valida la misma cadena de migraciones que define el producto y puede comprobar rowversion real.

Respawn

Respawn borra datos entre tests, pero se configura para ignorar __EFMigrationsHistory. Primero se aplican las migraciones y después se crea el Respawner.

Respawn no sustituye la preparación de esquema; su función es devolver los datos a un estado limpio con rapidez y determinismo.

WebApplicationFactory

La API mínima expone endpoints reales y WebApplicationFactory crea un cliente HTTP de integración. La cadena de conexión se sustituye por la base del fixture.

La prueba POST+GET valida routing, binding, DI, EF Core y SQL Server en una sola trayectoria, sin tocar una base de desarrollo del alumno.

Ejemplo ejecutable del concepto

```
public async Task InitializeAsync()
{
    await using var context = new AceriaDbContext(CreateOptions());
    await context.Database.MigrateAsync();
    await using var connection = new SqlConnection(ConnectionString);
    await connection.OpenAsync();
    _resawner = await Resawner.CreateAsync(connection, new ResawnerOptions
    {
        DbAdapter = DbAdapter.SqlServer,
        TablesToIgnore = [new Respawn.Graph.Table("__EFMigrationsHistory")]
    });
}
```

CSHARP

El ejemplo refleja la misma línea de implementación que usa el estado validado del proyecto. La práctica trabaja con el archivo real y comprueba el resultado en SQL Server LocalDB.

Profundización y contexto de uso

Profundización 1

Los tests más baratos comprueban lógica pura y colaboraciones; los de integración de proveedor verifican comportamiento que depende de la base; los HTTP recorren más capas y son más costosos. La suite debe colocar cada afirmación en el nivel más pequeño que pueda demostrarla fielmente.

Profundización 2

Usar el modelo completo de AceriaData sobre SQLite introduciría incompatibilidades específicas de SQL Server que distraen del objetivo. Por eso la prueba de comportamiento define un modelo mínimo Parent/Child y una FK.

Profundización 3

Crear una base LocalDB con un sufijo aleatorio evita colisiones con desarrollo y entre ejecuciones. La colección de xUnit comparte el fixture cuando conviene reutilizar el coste de crear esquema, mientras Respawn devuelve los datos a un estado limpio.

Profundización 4

Respawn debe borrar datos de negocio, no el registro que describe cómo se construyó el esquema. Si eliminara el historial sin reconstruir la base, EF Core podría creer que faltan migraciones aunque las tablas ya existan.

Mini caso de aplicación

Una regla de negocio puede probarse con Moq sin base. Una FK necesita un proveedor relacional. Un conflicto rowversion necesita SQL Server. Un endpoint necesita el host HTTP. Colocar cada prueba en el nivel adecuado evita falsos positivos y suites innecesariamente lentas.

Patrón de diagnóstico y preguntas de revisión

La técnica se revisa partiendo de una hipótesis concreta, una evidencia observable y una comprobación del estado final. Antes de convertir una recomendación en regla general conviene responder:

- ¿La afirmación del test depende del proveedor relacional?
- ¿Depende específicamente de SQL Server?
- ¿El esquema se creó mediante las mismas migraciones de la aplicación?
- ¿Respawn preserva la tabla de historial?
- ¿Cada caso empieza con datos conocidos?
- ¿El test HTTP comprueba composición de servicios y endpoint real?

Decisiones y trade-offs

Opción	Qué aporta	Trade-off
--------	------------	-----------

Unit + Moq	Sin base	Rápido; no valida SQL.
InMemory	Proveedor no relacional	Puede ocultar restricciones.
SQLite	Relacional ligero	Aplica reglas relacionales, pero no equivale a SQL Server.
LocalDB	Proveedor objetivo	Más fidelidad; más coste de ejecución.

Errores frecuentes

- Usar EnsureCreated en lugar de la cadena real de migraciones.
- Concluir que SQLite valida rowversion de SQL Server.
- Usar InMemory para validar restricciones relacionales.
- Limpiar la tabla `__EFMigrationsHistory` con Respawn.

Diseñar una pirámide de pruebas para persistencia

La pregunta determina el tipo de prueba

Si la pregunta es "¿el caso de uso llama al repositorio una vez?", no hace falta SQL Server. Si la pregunta es "¿una FK impide insertar un hijo huérfano?", hace falta semántica relacional. Si la pregunta es "¿rowversion de SQL Server cambia y detecta una copia obsoleta?", el proveedor SQL Server forma parte de la especificación. Seleccionar el nivel más pequeño que pueda responder la pregunta mantiene la suite rápida sin sacrificar confianza.

Esta separación evita una falsa dicotomía entre "todo unitario" y "todo integración". Una solución profesional necesita varias capas de prueba.

InMemory como doble específico

El proveedor InMemory no es un motor relacional. No debe usarse para demostrar restricciones, SQL, transacciones o traducción de consultas como si fueran SQL Server. Su utilidad está en escenarios donde ese comportamiento no forma parte de lo que se valida. AceriaData lo muestra aceptando un hijo huérfano que un proveedor relacional rechazaría.

Que una prueba pase con InMemory y falle con SQL Server puede revelar precisamente que la prueba dependía de semántica relacional no cubierta por el doble.

SQLite in-memory

SQLite es relacional y puede imponer claves foráneas cuando se configura, pero sigue siendo otro proveedor: dialecto, tipos, funciones, collations y generación de valores difieren. La metadata puede marcar una propiedad como token y `ValueGenerated.OnAddOrUpdate`, pero eso no convierte un `BLOB` de SQLite en el `rowversion` autogenerado de SQL Server.

Por eso AceriaData reserva la prueba de concurrencia de rowversion para LocalDB.

Migraciones reales y base aislada

La fixture de integración crea un nombre de base distinto, ejecuta `MigrateAsync` y conserva la cadena oficial de migraciones. Esto valida que el modelo no solo funciona cuando se crea un esquema ad hoc, sino que la historia de migraciones puede construir el estado esperado.

Respawn limpia datos entre tests sin borrar `__EFMigrationsHistory`. De esta forma cada caso parte de datos controlados sin reconstruir innecesariamente toda la base. La limpieza explícita también reduce pruebas que solo pasan por el orden accidental de ejecución.

WebApplicationFactory

Una prueba HTTP con `WebApplicationFactory` arranca el host de ASP.NET Core, sustituye la conexión por la base de pruebas y envía peticiones reales a endpoints. Esto cubre routing, serialización, DI, Infraestructure y SQL Server en el mismo flujo. Es más costoso que un unit test, por lo que se reserva para comportamientos que necesitan esa integración.

Lista de comprobación conceptual

- Escribir primero qué comportamiento se quiere probar.

- Elegir el proveedor mínimo que reproduce ese comportamiento.
- No usar InMemory como prueba de restricciones relacionales.
- No usar SQLite como prueba de `rowversion` de SQL Server.
- Aplicar las migraciones oficiales en integración.
- Aislar y limpiar datos para evitar dependencia entre tests.

Escenario de decisión: una prueba pasa en memoria y falla en SQL Server

Una prueba guarda un hijo cuyo `ParentId` no existe. Con InMemory puede completarse porque no se está usando un motor relacional que imponga la FK. Con SQLite configurado con claves foráneas, el mismo concepto produce un error relacional. Esa diferencia no convierte a InMemory en un proveedor "malo"; demuestra que no puede responder esa pregunta concreta.

Otro test intenta validar `rowversion`. SQLite puede representar metadata de concurrencia y un BLOB, pero no reproduce la columna autogenerada de SQL Server. Para esa afirmación, LocalDB es parte de la prueba. Dos contextos leen la misma fila, el primero guarda y el segundo debe recibir `DbUpdateConcurrencyException`.

Las pruebas HTTP añaden otra capa: `WebApplicationFactory` arranca la API, resuelve dependencias y habla con la base de pruebas. Si un error solo aparece en ese nivel, la causa puede estar en routing, serialización, DI o integración con el proveedor, no necesariamente en la regla de negocio aislada.

Anclaje en AceriaData

El concepto se implementa en `M05/PROYECTO0/5.11`. La carpeta contiene su propia `AceriaData.sln` y continúa directamente desde el punto anterior. La práctica restaura, compila y ejecuta ese estado completo; cuando el punto no cambia el modelo, también se comprueba que no existan cambios de modelo pendientes.

Resumen de la teoría

- Separar unit tests, provider-behavior tests e integration tests.
- Usar xUnit y Moq para probar Application sin EF Core.
- Entender las limitaciones de InMemory y SQLite como sustitutos de SQL Server.
- Ejecutar migraciones reales sobre SQL Server LocalDB.
- Aislar datos con Respawn sin borrar `__EFMigrationsHistory`.
- Validar `rowversion` real de SQL Server.

Punto 5.12 — Buenas prácticas y anti-patrones en persistencia empresarial

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL, sin experiencia previa en ORMs ni en Entity Framework Core.

Proyecto: Este punto continúa el proyecto acumulativo AceriaData y estudia buenas prácticas y anti-patrones en persistencia empresarial sobre .NET 8, Entity Framework Core 8 y SQL Server LocalDB.

Objetivos de aprendizaje

- Identificar anti-patrones mediante evidencia observable y no por etiquetas.
- Reproducir N+1 y refactorizarlo con Include y proyección según necesidad.
- Comparar roundtrips, tracking, columnas y equivalencia funcional.
- Demostrar el over-fetching y su refactorización.
- Comprobar el fallo de traducción de un método .NET dentro de Where en EF Core 8.
- Evitar reglas absolutas sobre Include, SplitQuery, Fluent API o Repository.
- Construir una matriz de síntoma, consecuencia, refactor y trade-off.

Consideraciones técnicas en EF Core 8

Las recomendaciones se validan mediante before/after y trade-offs; no se convierten Include, SplitQuery, Fluent API o Repository en reglas absolutas.

Desarrollo teórico

Anti-patrón como evidencia

Un anti-patrón no se demuestra escribiendo una lista. Debe existir un síntoma observable, una consecuencia y una alternativa cuyo resultado funcional pueda compararse.

5.12 reutiliza el interceptor de comandos, ChangeTracker y ToQueryString para convertir recomendaciones en evidencias.

N+1 before/after

El escenario base carga cabeceras y luego ejecuta Count para las planchas de cada orden. El número de roundtrips crece con el número de órdenes.

Incluye reduce el patrón cuando realmente se necesitan entidades relacionadas. Una proyección puede ser aún más apropiada cuando solo se necesita TotalPlanchas. Las tres variantes deben devolver resultados equivalentes.

Over-fetching

Cargar OrdenFabricacion completa con tracking materializa todas sus propiedades, incluida RowVersion. La proyección del laboratorio recupera solo NumeroOrden, Cliente, Estado y FechaCreacion con AsNoTracking.

El programa compara columnas del modelo, presencia de RowVersion en el SQL, entradas en ChangeTracker y equivalencia del resumen.

Método no traducible en Where

En EF Core 8 un método .NET arbitrario dentro del predicado de Where normalmente provoca InvalidOperationException porque no puede traducirse a SQL. No se realiza silenciosamente una evaluación cliente del filtro.

Si la aplicación cruza explícitamente a AsEnumerable, el filtro sí puede ejecutarse en cliente, pero primero se transfieren las filas que haya producido la parte servidor. El trade-off debe ser visible.

Reglas no absolutas

Data Annotations no son intrínsecamente un anti-patrón; Fluent API ofrece mayor capacidad y separación. AsSplitQuery no es automáticamente mejor con varias colecciones; cambia roundtrips y consistencia temporal. Repository tampoco es obligatorio.

Las guías técnicas deben conservar el contexto que hace válida una recomendación.

Matriz de decisión

El cierre del módulo registra para cada caso: síntoma/evidencia, consecuencia, refactor y trade-off. Incluye ciclo de vida de DbContext, N+1, over-fetching, SplitQuery, traducción, sargabilidad, configuración, Repository, migraciones y testing.

El objetivo final no es memorizar prohibiciones, sino aprender a formular una hipótesis, observar SQL/estado y comprobar que el refactor mantiene el comportamiento de negocio.

Ejemplo ejecutable del concepto

```
var consultaProyectada = _context.OrdenesFabricacion
    .AsNoTracking()
    .Where(o => o.Estado == "Pendiente")
    .OrderBy(o => o.Id)
    .Select(o => new
    {
        o.NumeroOrden, o.Cliente, o.Estado, o.FechaCreacion
    });
var sqlProyectada = consultaProyectada.ToQueryString();
var proyectadas = consultaProyectada.ToList();
```

CSHARP

El ejemplo refleja la misma línea de implementación que usa el estado validado del proyecto. La práctica trabaja con el archivo real y comprueba el resultado en SQL Server LocalDB.

Profundización y contexto de uso

Profundización 1

La matriz final relaciona caso, síntoma, consecuencia, refactor y trade-off. Esa estructura obliga a explicar por qué algo es problemático. DbContext largo no es solo una etiqueta: el síntoma puede ser crecimiento del ChangeTracker y estado obsoleto.

Profundización 2

Una versión optimizada que devuelve un resultado diferente no es una mejora equivalente. Por eso el diagnóstico de N+1 compara número de órdenes y planchas entre la versión original, Include y proyección antes de comparar roundtrips y tracking.

Profundización 3

El ejemplo de over-fetching obtiene del modelo de EF el número de propiedades de OrdenFabricacion y compara ese shape con una proyección de cuatro campos. Además usa ToQueryString para comprobar que RowVersion aparece en la entidad completa y no en la proyección.

Profundización 4

AsNoTracking reduce trabajo cuando los objetos solo se leen, pero desactivar tracking en una operación que después modifica entidades puede obligar a adjuntarlas y gestionar estado manualmente. El criterio es si el resultado necesita participar en una unidad de escritura.

Mini caso de aplicación

Una pantalla solo muestra número, cliente, estado y fecha. Cargar OrdenFabricacion completa con tracking funciona, pero materializa más columnas y mantiene estado que la pantalla no modifica. Proyectar cuatro campos conserva el resultado útil con menos trabajo observable.

Patrón de diagnóstico y preguntas de revisión

La técnica se revisa partiendo de una hipótesis concreta, una evidencia observable y una comprobación del estado final. Antes de convertir una recomendación en regla general conviene responder:

- ¿La versión nueva devuelve exactamente el mismo resultado que necesita el consumidor?
- ¿Cuántos comandos ejecuta cada alternativa?
- ¿Qué entidades quedan rastreadas?
- ¿Qué columnas aparecen en el SQL?
- ¿La alternativa introduce otros trade-offs como varios roundtrips?
- ¿La regla propuesta debe expresarse como decisión contextual?

Decisiones y trade-offs

Opción	Qué aporta	Trade-off
--------	------------	-----------

Include	Materializa relacionados	Útil cuando el resultado necesita entidades completas.
Proyección	Shape reducido	Reduce columnas/materialización cuando se necesita un resumen.
SplitQuery	Divide consultas	Puede reducir explosión cartesiana, pero aumenta roundtrips.
Tracking	Necesario para modificar cómodamente	No debe eliminarse por dogma en operaciones de escritura.

Errores frecuentes

- Aplicar Include como solución universal al N+1.
- Aplicar AsSplitQuery automáticamente.
- Decir que Data Annotations son un anti-patrón por sí mismas.
- Afirmar que un método no traducible dentro de Where se filtra silenciosamente en cliente.

Convertir buenas prácticas en decisiones demostrables

Empezar por equivalencia funcional

Antes de afirmar que una versión es mejor que otra, ambas deben entregar el mismo resultado que necesita el consumidor. En el ejemplo N+1, la consulta inicial más las consultas por orden, el `Include` y la proyección calculan el mismo número de órdenes y planchas. Solo después tiene sentido comparar comandos, tracking y forma de SQL.

Esta disciplina evita optimizaciones que cambian silenciosamente la semántica. Reducir consultas no sirve si se dejan de incluir datos que el caso de uso necesita.

N+1: reconocer la forma del problema

El patrón aparece cuando se obtiene una colección principal y después se emite una consulta adicional por cada elemento. El contador de comandos lo hace visible. `Include` puede resolverlo cuando la aplicación necesita las entidades relacionadas; una proyección con un agregado puede ser más apropiada cuando solo se necesita un resumen.

Aplicar `Include` por defecto puede provocar over-fetching. Aplicar proyección por defecto puede ser incómodo si después se necesita modificar el agregado. La forma de salida determina la técnica.

Over-fetching: observar columnas y tracking

AceriaData compara una entidad completa con tracking y una proyección de cuatro campos `AsNoTracking`. Se cuentan propiedades del modelo, se inspecciona el SQL y se comprueba que `RowVersion` desaparece de la proyección. También se observa cuántas entidades quedan en el tracker.

No se concluye que el tracking sea un anti-patrón. Cuando el objetivo es editar y guardar la entidad, el tracking ofrece precisamente el servicio que se necesita.

Traducción y frontera cliente/servidor

En EF Core 8, un método `.NET` no traducible dentro de un `Where` no se evalúa silenciosamente en cliente. La consulta falla antes de emitir SQL. Si la aplicación llama a `AsEnumerable`, cruza de forma explícita la frontera y el predicado posterior se ejecuta en `.NET`.

La explicitud es valiosa porque obliga a reconocer que pueden transferirse más filas. La decisión se toma con conocimiento de cardinalidad y coste, no como solución automática a un error de traducción.

Reglas contextuales

Data Annotations y Fluent API son mecanismos válidos con distintas capacidades y grados de centralización. `AsSplitQuery` puede reducir explosión cartesiana, pero añade roundtrips y puede observar estados temporales distintos entre consultas. Aplicar funciones sobre columnas puede afectar sargabilidad según expresión, collation, índice y proveedor; no existe una ley de "función igual a índice inútil".

La matriz de AceriaData termina cada regla con un trade-off. Ese formato es más útil que una lista de prohibiciones porque obliga a describir cuándo una técnica ayuda y qué coste introduce.

Lista de comprobación conceptual

- Demostrar equivalencia funcional antes del before/after.
- Contar comandos para identificar N+1.
- Inspeccionar columnas del SQL para identificar over-fetching.
- Observar tracking cuando el caso de uso es solo lectura.
- Hacer explícita cualquier frontera de evaluación cliente.
- Formular recomendaciones con su contexto y trade-off.

Escenario de decisión: optimizar una lista de órdenes

Una vista necesita número de orden, cliente, estado, fecha y cantidad de planchas. El enfoque N+1 carga primero las órdenes y después consulta el recuento de planchas para cada una. Funciona, pero el contador revela una forma de ejecución cuyo número de comandos crece con las órdenes. Un `Include` puede reducir roundtrips, aunque materializa las planchas completas. Una proyección puede pedir directamente el agregado y las columnas del resumen.

Las tres variantes deben compararse con el mismo resultado funcional. Solo entonces se interpretan número de comandos, SQL y tracking. Esta secuencia impide declarar vencedora una consulta que simplemente devuelve menos información de la requerida.

El mismo criterio se aplica al over-fetching. La entidad completa con tracking es adecuada cuando se va a editar; la proyección `AsNoTracking` es adecuada para una vista de lectura. La "buena práctica" no es eliminar tracking, sino alinear materialización y seguimiento con el uso real de los datos. La matriz final convierte estas observaciones en decisiones con contexto, no en prohibiciones universales.

Anclaje en AceriaData

El concepto se implementa en `M05/PROYECTO0/5.12`. La carpeta contiene su propia `AceriaData.sln` y continúa directamente desde el punto anterior. La práctica restaura, compila y ejecuta ese estado completo; cuando el punto no cambia el modelo, también se comprueba que no existan cambios de modelo pendientes.

Resumen de la teoría

- Identificar anti-patrones mediante evidencia observable y no por etiquetas.
- Reproducir N+1 y refactorizarlo con Include y proyección según necesidad.
- Comparar roundtrips, tracking, columnas y equivalencia funcional.
- Demostrar el over-fetching y su refactorización.
- Comprobar el fallo de traducción de un método .NET dentro de Where en EF Core 8.
- Evitar reglas absolutas sobre Include, SplitQuery, Fluent API o Repository.

Lectura transversal del Módulo 5

Los doce puntos forman una secuencia de decisiones conectadas. Concurrencia responde a qué ocurre cuando el estado cambia entre lectura y escritura. Transacciones responden a qué cambios deben confirmarse juntos. Migraciones responden a cómo evoluciona el esquema sin perder control operacional. Logging y testing responden a cómo se demuestra que el sistema hace lo esperado. Las buenas prácticas del cierre sirven para evitar que una solución local cree problemas en otra dimensión.

De la concurrencia a la transacción

Un token de concurrencia no sustituye una transacción y una transacción no sustituye un token. La transacción controla atomicidad y aislamiento de una unidad de trabajo; el token compara la versión que el actor leyó con la que existe al escribir. Una operación puede necesitar ambos mecanismos.

Del despliegue al trabajo en equipo

Una migración empieza como código generado en una rama y termina como cambio compartido de base de datos. Antes de publicarse, una migración propia puede regenerarse para incorporar cambios de otros desarrolladores. Después de aplicarse en un entorno compartido, reescribirla cambia la historia que otras bases ya conocen.

De la arquitectura al testing

Repository y Unit of Work se introducen como una frontera elegida de AceriaData. Esa frontera facilita unit tests de Application porque los casos de uso reciben interfaces propias. Sin embargo, ocultar EF Core no elimina la necesidad de probar EF Core: consultas, migraciones, relaciones y tokens siguen necesitando integración contra un proveedor real.

De observabilidad a evidencia

Logging, DiagnosticSource y EventCounters hacen visible el comportamiento durante la ejecución. Los tests convierten determinadas expectativas en comprobaciones repetibles. Ambos enfoques se complementan y deben respetar límites de volumen, coste y datos sensibles.

Cinco preguntas para cualquier decisión de persistencia

1. **¿Qué resultado funcional debe mantenerse?** Antes de optimizar, abstraer o reintentar, se define qué significa que la operación sea correcta.
2. **¿Qué comportamiento depende del proveedor?** rowversion, SQL generado, collations, transacciones y migraciones necesitan validación en el motor correspondiente.
3. **¿Qué estado se comparte y durante cuánto tiempo?** Esta pregunta afecta a DbContext, transacciones, scopes y ediciones concurrentes.
4. **¿Qué evidencia demostrará la afirmación?** Puede ser estado final, SQL, roundtrips, tracking, historial, excepción o respuesta HTTP.
5. **¿Qué trade-off introduce la solución?** Include materializa más datos, Split Query añade roundtrips, Repository añade abstracción y una prueba de mayor alcance suele costar más tiempo de ejecución.

Casos integradores de persistencia empresarial

Caso integrador 1: edición concurrente y despliegue del token

Una aplicación ya está en producción y necesita empezar a detectar ediciones concurrentes de órdenes. El cambio no termina al añadir `RowVersion` a la clase. Primero se configura el modelo, se genera una migración que represente el cambio, se revisa el SQL de despliegue y se prueba sobre una base aislada. Después de desplegar, dos contextos independientes deben reproducir un conflicto real y el equipo debe decidir qué política aplicará la interfaz cuando aparezca `DbUpdateConcurrencyException`.

El caso conecta 5.1, 5.2, 5.3 y 5.6. La concurrencia define la semántica que se quiere proteger; la migración introduce el soporte físico; la política de resolución determina la experiencia después del conflicto; el proceso de despliegue garantiza que aplicación y base evolucionen de forma coordinada. Si cualquiera de esas piezas falta, el mecanismo puede estar configurado en código pero no ser operativo en el entorno final.

Caso integrador 2: una operación escribe base de datos y publica un evento

Una orden se confirma y, a continuación, otro sistema debe ser informado. Una transacción de SQL Server puede proteger los cambios de la base, pero un efecto externo ordinario no se revierte automáticamente con `TransactionScope`. Si el proceso escribe primero en SQL y luego publica fuera, hay que razonar qué ocurre cuando falla cada paso y cómo se recupera una repetición.

El caso conecta 5.4 y 5.5 con la observabilidad de 5.10. La frontera transaccional debe quedar explícita y los logs deben permitir reconstruir qué etapa se completó. Si se adopta un mecanismo de consistencia como outbox, también tendrá que probarse con fallos parciales y operaciones repetidas. La lección es que atomicidad local y consistencia entre sistemas son problemas relacionados, pero no idénticos.

Caso integrador 3: varias ramas y varias bases

Dos equipos desarrollan cambios de esquema en paralelo mientras existen entornos que no están todos en la misma migración. La resolución empieza en 5.8: se reconstruye el estado común de las ramas y se regenera una migración todavía local cuando procede. Después, 5.7 aporta un script idempotente que puede reconocer qué migraciones faltan en cada base, y 5.6 aporta el proceso de revisión, ejecución y verificación.

La combinación obliga a conservar dos historias coherentes: la historia de Git y la historia de `EFMigrationsHistory`. Un merge de texto que compile no demuestra que ambas historias coincidan. La validación final aplica la cadena a una base aislada, comprueba que no quedan cambios de modelo pendientes y verifica los objetos del esquema que representan los cambios de ambos equipos.

Caso integrador 4: una optimización que necesita pruebas y telemetría

Una pantalla de lectura genera muchas consultas. El diagnóstico de 5.12 identifica N+1 contando comandos y propone una proyección que conserva el mismo resultado funcional. Antes de desplegarla, 5.11 permite crear pruebas que comparen resultados y comportamiento del proveedor cuando sea necesario. Después del despliegue, 5.10 ofrece logs y métricas para observar si la forma de ejecución real coincide con la esperada bajo carga.

La secuencia evita dos errores frecuentes: optimizar solo por intuición y considerar suficiente una prueba aislada de tiempo. Primero se demuestra la forma del problema —roundtrips, SQL, columnas, tracking—, después se protege el comportamiento mediante pruebas y finalmente se observa el sistema ejecutándose. Si la carga real muestra otra limitación, la siguiente mejora parte de evidencia nueva y no de una regla aplicada automáticamente.

Método de trabajo que resume el módulo

Ante un problema de persistencia, el orden de razonamiento puede resumirse en cinco movimientos. Primero se define la semántica correcta: qué datos deben quedar y qué conflictos o estados parciales son aceptables. Segundo se identifica la frontera: `DbContext`, transacción, migración, repositorio, prueba o proceso de despliegue. Tercero se obtiene evidencia del proveedor cuando la afirmación depende de SQL Server. Cuarto se implementa la solución mínima que preserve la semántica. Quinto se verifica el resultado y se documenta el coste o trade-off introducido.

Este método sirve tanto para una excepción de concurrencia como para una migración divergente o una consulta N+1. Cambian las herramientas, pero no la disciplina: reproducir, observar, decidir, comprobar y conservar una explicación que pueda volver a ejecutarse.

Matriz de evidencia por problema

Una afirmación sobre persistencia es más útil cuando se vincula a la evidencia capaz de confirmarla. Para una actualización perdida, la evidencia principal es el estado final junto con los `UPDATE` ejecutados. Para un conflicto con `rowversion`, se añade la excepción y la condición de concurrencia del `WHERE`. Para una transacción, se verifica que datos existen después de `Commit`, `Rollback` o `RollbackToSavepoint`. Para una migración, se comparan historial y esquema. Para una optimización, se exige primero equivalencia funcional y después se observan comandos, columnas, tracking o forma de consulta.

Esta relación evita usar una herramienta porque está disponible en lugar de porque responde a la pregunta. `ToQueryString()` es excelente para estudiar la representación SQL de una consulta LINQ, pero no demuestra que la consulta haya sido ejecutada. El logging o un interceptor sí permiten observar comandos emitidos. Un test unitario con Moq demuestra interacción con una abstracción, pero no puede afirmar cómo SQL Server interpreta una FK. Un test de integración puede responder esa segunda pregunta, aunque cueste más preparación.

También conviene distinguir evidencia de diagnóstico y evidencia de aceptación. Un `EventCounter` puede señalar que el número de consultas activas cambió y orientar una investigación. Para demostrar que una refactorización de N+1 conserva el resultado, hace falta además comparar los datos producidos. De forma análoga, que una migración aparezca en `EFMigrationsHistory` es necesario, pero si existe sospecha de drift se comprueba también el objeto físico que debería haber creado.

Problema	Evidencia primaria	Evidencia complementaria
Edición concurrente sin token	Estado final después de dos escrituras	Comandos SQL observados
Conflicto con token	<code>DbUpdateConcurrencyException</code>	Token original/actual y <code>WHERE</code> del <code>UPDATE</code>
Resolución de conflicto	Valores finales por propiedad	Número de intentos y lecturas adicionales
Atomicidad	Filas persistidas tras éxito o fallo	Estado de transacción y comandos
Migración de producción	Historial y esquema esperado	Artefacto SQL/bundle y smoke test
Script idempotente	Segunda ejecución sin duplicar historia	Verificación del esquema tras ambos pases
Migraciones en equipo	Snapshot/modelo fusionado coherente	Base limpia + <code>has-pending-model-changes</code>
Repository/UoW	Responsabilidades y resultado funcional	Unit tests sobre interfaces
Observabilidad	Eventos/logs/contadores realmente producidos	Correlación y recepción del exportador
Testing de proveedor	Resultado en el motor que define la semántica	Dobles ligeros para casos no dependientes del motor
N+1 / over-fetching	Roundtrips, SQL, columnas y tracking	Medición de rendimiento bajo carga representativa

La matriz no pretende convertir el diagnóstico en una receta rígida. Su función es recordar que distintos problemas requieren distintos observables. Si una conclusión depende del proveedor, el proveedor debe aparecer en la prueba. Si depende de la experiencia de usuario, el estado funcional debe verificarse. Si depende del rendimiento, primero se elimina cualquier diferencia funcional y después se diseña una medición repetible. De este modo el módulo termina con un criterio común para concurrencia, transacciones, despliegue, arquitectura y optimización.

Glosario operativo del módulo

Token de concurrencia. Valor que EF Core conserva como parte del estado original y utiliza para comprobar que una fila sigue en la versión esperada al escribir.

Estado original. Valores asociados a la entidad cuando comienza a rastrearse; sirven como referencia para concurrencia.

Estado actual. Valores que la aplicación quiere persistir; tras un conflicto pueden diferir del estado original y del estado de base.

Savepoint. Marca dentro de una transacción que permite volver a un punto intermedio sin crear una segunda transacción independiente.

Transacción ambiental. Transacción accesible mediante `Transaction.Current` a la que pueden adherirse recursos compatibles.

Promoción distribuida. Cambio desde una transacción local a coordinación distribuida cuando participan recursos que no pueden resolverse dentro de una sola transacción local.

Migration bundle. Ejecutable generado por EF Core para aplicar la cadena de migraciones sin necesitar la CLI de `dotnet ef` instalada en el destino.

Script idempotente. SQL que consulta el historial de migraciones y ejecuta solo los cambios aún no aplicados.

Snapshot del modelo. Representación que EF Core usa para calcular el siguiente delta de migración.

Respawn. Herramienta usada en tests de integración para devolver datos a un estado limpio sin reconstruir el esquema en cada caso.

WebApplicationFactory. Infraestructura de ASP.NET Core para arrancar una aplicación dentro del proceso de pruebas y enviar peticiones HTTP al host real.

DiagnosticListener. API de diagnóstico basada en publicación/suscripción; EF Core expone eventos que un observador puede filtrar y procesar.

EventCounters. Contadores publicados mediante EventSource que permiten observar métricas agregadas del proceso.

N+1. Patrón en el que una consulta inicial provoca después una consulta adicional por cada elemento principal.

Over-fetching. Transferencia o materialización de más datos de los necesarios para el caso de uso.

Sargabilidad. Capacidad de un predicado para aprovechar de forma eficiente las estructuras de acceso del motor; depende de expresión, proveedor, collation y diseño físico.

Equivalencia funcional. Condición previa a comparar dos implementaciones como alternativas de rendimiento: ambas deben producir el mismo resultado requerido por el caso de uso.

Resumen de la teoría

El módulo 5 conecta concurrencia, transacciones, migraciones, arquitectura, observabilidad y testing como partes de un mismo problema: construir una persistencia que mantenga semántica de negocio, pueda desplegarse y pueda demostrarse. La disciplina central consiste en definir el comportamiento esperado, observar el proveedor real cuando la afirmación depende de él, aplicar la técnica más pequeña que resuelva el problema y documentar el trade-off.